ABSTRACT

The Gym Management System is a comprehensive desktop application designed to streamline and automate the day-to-day operations of a fitness center. Built using the PyQt5 GUI framework and Python-based application logic, the system provides an intuitive and modern user interface combined with robust backend functionality.

The application follows a layered architecture based on the Model-View-Controller (MVC) design pattern, ensuring scalability, maintainability, and clear separation of concerns. It consists of four primary layers: the Presentation Layer for interactive user interfaces, the Business Logic Layer for core application processing, the Data Access Layer for efficient database communication, and the Database Layer powered by SQLite for reliable data storage.

Key features include member and trainer management, workout plan creation and assignment, membership handling, payment tracking, equipment inventory management, and real-time analytics dashboards. Advanced functionalities such as BMI calculation, progress tracking, and data visualization using Matplotlib enhance decision-making and user experience.

The system ensures data integrity and consistency through ACID-compliant transactions, normalized database design (3NF), and optimized queries. Its modular and extensible structure makes it suitable for small to medium-sized gyms, while also allowing future enhancements such as cloud integration or mobile support.

Overall, the Gym Management System demonstrates a professional-grade software solution that combines usability, performance, and modern software engineering practices to efficiently manage fitness center operations.

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	ER DIAGRAM OF GYM MANAGEMENT SYSTEM	8
3.2	USE CASE DIAGRAM OF GYM MANAGEMENT SYSTEM	9
3.3	ARCHITECTURE DIAGRAM OF GYM MANAGEMENT SYSTEM	10
5.1	LOGIN	78
5.2	DASHBOARD	78
5.3	MEMBER MANAGEMENT	79
5.4	TRAINER MANAGEMENT	79
5.5	BODY METRICS TRACKING	80
5.6	PROGESS CHART	80
5.7	WORKOUT PLANS MANAGEMENT	81
5.8	PAYMENTS MANAGEMENT	81
5.9	MEMBERSHIP PLANS MANAGEMENT	82
5.10	EQUIPMENT MANAGEMENT	82
5.11	REAL-TIME ANALYTICS DASHBOARD	83

LIST OF ABBREVIATIONS

GUI	Graphical User Interface
DB	Database
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
MVC	Model-View-Controller
UI	User Interface
API	Application Programming Interface
OOP	Object-Oriented Programming
BMI	Body Mass Index
ID	Identification
DOB	Date of Birth
PyQt	Python Qt Framework
DBMS	Database Management System
ER	Entity Relationship
FK	Foreign Key
PK	Primary Key

CHAPTER 1

INTRODUCTION

1.1 Introduction:

The **Gym Management System** is a desktop-based application designed to manage and automate the daily operations of a gym. Traditional gym management often involves manual record-keeping, which can be time-consuming, error-prone, and inefficient. This system provides a digital solution to handle various activities such as member registration, trainer management, workout planning, and payment tracking.

Developed using **Python** and the **PyQt5 framework**, the application offers a user-friendly graphical interface that simplifies interaction for gym staff. The system also uses a structured database to store and manage data securely and efficiently. By integrating different functionalities into a single platform, the system improves productivity and reduces administrative workload.

1.2 Problem statement:

Many gyms still rely on manual methods such as paper records or basic spreadsheets to manage their operations, which often leads to inefficiencies and errors. Maintaining and updating large volumes of data becomes difficult, and there is a higher risk of human mistakes and data redundancy. Additionally, manual systems lack real-time tracking of memberships and payments, making it harder to monitor active members and financial transactions. Managing trainers, scheduling workout plans, and organizing records also becomes time-consuming and less effective. Furthermore, generating reports and gaining meaningful insights from the data is limited. Therefore, there is a need for a reliable and automated system that can efficiently manage gym operations and improve overall performance.

1.3 Objective:

The primary objective of the Gym Management System is to design and develop an efficient, reliable, and user-friendly application that simplifies and automates the core operations of a gym. The system focuses on improving data management, reducing manual effort, and enhancing overall efficiency. The key objectives of the system are as follows:

- To automate the process of member registration and maintain accurate member records
- To manage trainer details, schedules, and their assignments effectively
- To create and assign workout plans based on individual member requirements
- To handle membership plans and track their validity and duration
- To record and monitor payment transactions in an organized manner
- To maintain and manage gym equipment and inventory details
- To track the fitness progress of members and maintain performance records
- To generate basic reports and analytics for better decision-making
- To reduce manual errors and improve data accuracy
- To provide a simple and user-friendly interface for easy operation

1.4 Scope and Motivation:

The scope of the Gym Management System is focused on providing an efficient and organized solution for managing the core operations of a gym or fitness center. The system is designed to handle various activities such as member registration, trainer management, membership plan handling, payment tracking, workout plan management, and equipment monitoring. It enables gym staff to store, update, and retrieve information in a structured manner, reducing the complexity associated with manual record keeping. The application is primarily intended for small to medium-sized gyms, where managing data manually becomes increasingly difficult as the number of members grows.

The system also includes features for tracking member fitness progress, such as maintaining records of body metrics and workout performance. In addition, it provides basic reporting and analytical capabilities that help gym owners understand membership trends, revenue generation, and overall performance. The modular design of the system ensures that each functionality operates independently while remaining integrated with other modules, making the system flexible and easy to maintain. Furthermore, the system can be extended in the future to include advanced features such as cloud-based data storage, mobile application integration, online booking systems, and automated notifications.

The motivation behind developing the Gym Management System arises from the limitations and challenges associated with traditional manual methods of gym management. Managing records using paper or basic spreadsheets is not only time-consuming but also prone to errors and inefficiencies. As gyms grow and the volume of data increases, it becomes difficult to maintain accuracy and consistency in records. Issues such as data redundancy, loss of information, and difficulty in retrieving records highlight the need for a more reliable and automated solution.

Another key motivation is to improve the overall efficiency and productivity of gym operations. By automating routine tasks such as data entry, record updates, and report generation, the system reduces the workload on staff and allows them to focus on more important activities, such as customer service and member engagement. The system also enhances accuracy by minimizing human errors and ensuring proper validation of data.

In addition, the need for better decision-making plays an important role in motivating the development of this system. With access to organized data and basic analytical reports, gym owners can make informed decisions regarding membership plans, pricing, resource allocation, and business growth strategies. The system also improves transparency and accountability by maintaining proper records of payments, memberships, and activities.

Overall, the Gym Management System is motivated by the need to replace inefficient manual processes with a modern, reliable, and user-friendly solution that enhances operational efficiency, improves data management, and supports the growth of fitness centers in a systematic manner.

CHAPTER 2

EXISTING SYSTEM

The existing system used in many gyms and fitness centers is largely based on manual processes and traditional methods of record keeping. Most gyms rely on physical registers, paper files, or basic spreadsheet tools to store and manage information related to members, trainers, payments, and workout schedules. While this approach may be sufficient for very small gyms, it becomes increasingly inefficient and difficult to manage as the number of members and operations grows.

In a manual system, all member details such as personal information, contact details, membership type, and joining date are recorded by hand or entered into simple spreadsheets. Similarly, trainer details, workout plans, and attendance records are maintained separately, often without proper integration. This lack of centralized data management leads to disorganization and makes it difficult to maintain consistency across records. For example, updating a member's information in one place may not reflect in other related records, causing confusion and errors.

One of the major drawbacks of the existing system is the time-consuming nature of data handling. Tasks such as adding new members, updating membership details, or checking payment status require manual effort and can take a significant amount of time. Retrieving specific information, such as a member's payment history or workout progress, involves searching through multiple files or records, which is both inefficient and frustrating. This reduces the productivity of staff and affects the overall efficiency of gym operations.

Another significant issue with manual systems is the high possibility of human errors. Mistakes such as incorrect data entry, duplication of records, missing information, or misplacement of files are common. These errors can lead to serious problems, including incorrect billing, loss of important data, and poor customer service. Since there is no automated validation or error-checking mechanism, maintaining data accuracy becomes a major challenge.

The existing system also lacks real-time tracking and monitoring capabilities. Gym staff cannot easily track membership expiry dates, payment dues, or attendance records without manually checking each record. This can result in missed renewals, delayed payments, and inefficient management of resources. Additionally, there is no proper way to monitor member progress, such as fitness improvements or workout

achievements, which limits the ability to provide personalized services.

Another limitation of the current system is the absence of proper reporting and analytics. Generating reports on revenue, membership growth, or trainer performance requires manual calculation and compilation of data. This process is not only time-consuming but also prone to errors. Without accurate and timely reports, gym management finds it difficult to analyze business performance and make informed decisions for improvement.

Security is also a concern in the existing system. Physical records can be easily lost, damaged, or accessed by unauthorized individuals. Even in the case of spreadsheets, there is limited protection against data loss or unauthorized access. Backup and recovery mechanisms are often not in place, increasing the risk of losing important information permanently.

Furthermore, managing equipment and inventory in a manual system is inefficient. Details about gym equipment, their condition, maintenance schedules, and availability are often not properly tracked. This can lead to poor maintenance, equipment failure, and increased operational costs. Similarly, managing trainer schedules and member assignments becomes complicated without a structured system.

The lack of integration between different functional areas is another major drawback. Member management, payment tracking, trainer scheduling, and workout planning are handled separately, leading to fragmented data and poor coordination. This makes it difficult to get a complete view of gym operations and affects decision-making.

In addition, manual systems do not provide a user-friendly interface or ease of access. Staff members need to spend more time learning and handling records manually, which increases workload and reduces efficiency. There is also no provision for automation, notifications, or reminders, such as alerts for membership expiry or pending payments, which are essential for smooth operation.

Overall, the existing system is outdated, inefficient, and not suitable for modern gym management requirements. It lacks automation, accuracy, security, and scalability. As gyms continue to grow and customer expectations increase, there is a strong need for a computerized system that can overcome these limitations. A digital solution can provide better data management, improved efficiency, real-time tracking, and enhanced decision-making capabilities, making it essential to replace the traditional existing system.

CHAPTER 3

DESIGN

3.1 Entity-Relationship (ER) Design:

The Entity-Relationship (E-R) design represents the logical structure of the database used in the Gym Management System. It defines the entities, their attributes, and the relationships between them. The database is designed to ensure data consistency, avoid redundancy, and support efficient data retrieval.

Entities in the System

The system consists of several key entities, each representing a real-world object in the gym environment:

- **Members**
  This entity stores information about gym members. Attributes include member ID, name, age, contact details, membership type, and joining date. It is one of the central entities in the system.
- **Trainers**
  This entity contains details about trainers, such as trainer ID, name, specialization, and contact information. Trainers are assigned to members for guidance and workout planning.
- **Memberships**
  This entity defines different membership plans offered by the gym, including plan name, price, and duration. It helps in managing subscriptions and tracking validity.
- **Payments**
  This entity records all payment transactions made by members. Attributes include payment ID, member ID, amount, payment date, and payment method.
- **Workout Plans**
  This entity stores details about workout routines assigned to members. It includes plan ID, member ID, trainer ID, goals, and duration.
- **Body Metrics**
  This entity tracks the physical progress of members, including weight, height, BMI, and date of measurement.
- **Equipment**
  This entity manages gym equipment details such as equipment ID, name, type, condition, and maintenance status.

Relationships Between Entities

The relationships between entities define how different components of the system are connected:

- A **member** can have one membership plan, but a membership plan can be assigned to many members (one-to-many relationship).
- A **member** can make multiple payments, but each payment is associated with a single member (one-to-many relationship).
- A **trainer** can train multiple members, while a member is usually assigned to one trainer (one-to-many relationship).
- A **member** can have multiple workout plans over time, but each workout plan belongs to one member (one-to-many relationship).
- A **trainer** can create multiple workout plans (one-to-many relationship).
- A **member** can have multiple body metric records, each representing progress over time (one-to-many relationship).

These relationships are implemented using foreign keys in the database to maintain referential integrity.

Database Design Principles

The database is designed following normalization principles, particularly up to the Third Normal Form (3NF), to eliminate redundancy and ensure data consistency. Each table is structured to store only relevant data, and dependencies are properly managed. Primary keys uniquely identify each record, while foreign keys establish relationships between tables.

Indexes are used to improve query performance, especially for frequently accessed data such as member details and payment records. Constraints are applied to ensure data validity, such as not allowing null values for essential fields.

Advantages of E-R Design

The E-R design provides a clear understanding of the system's data structure and relationships. It helps developers implement the database efficiently and ensures that the system can handle data operations effectively. It also makes it easier to modify or extend the system in the future.

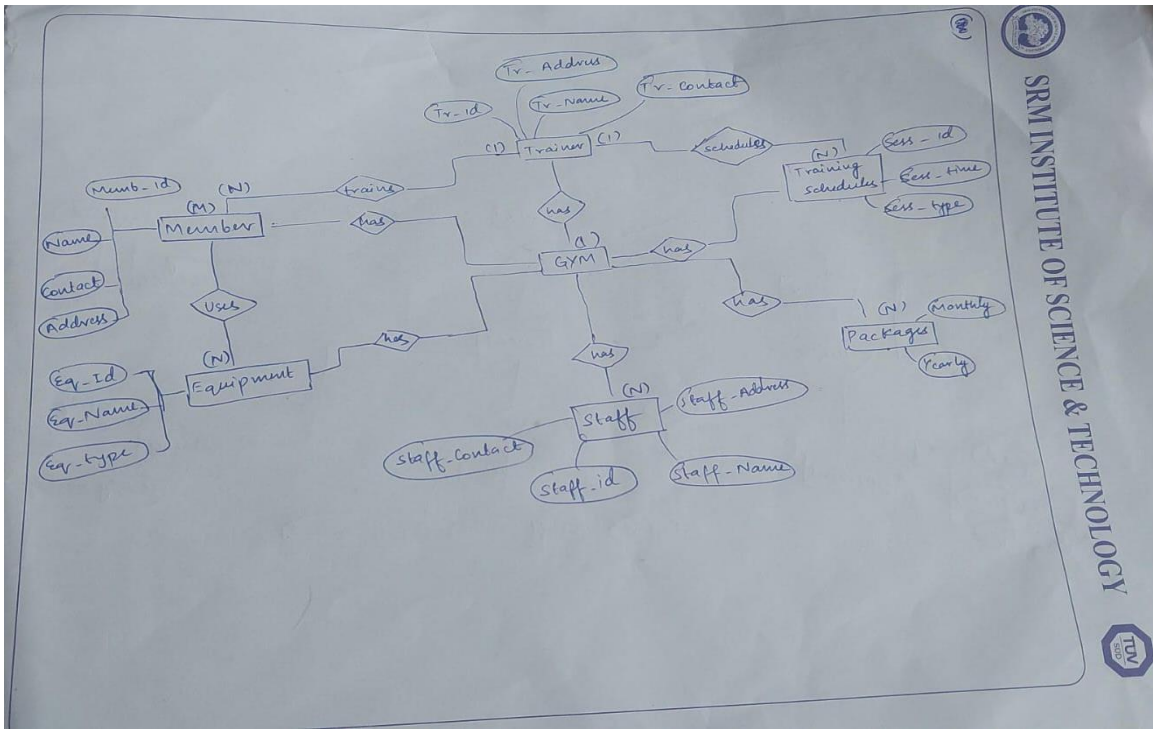


Fig. 3.1 ER Diagram of GYM Management System.

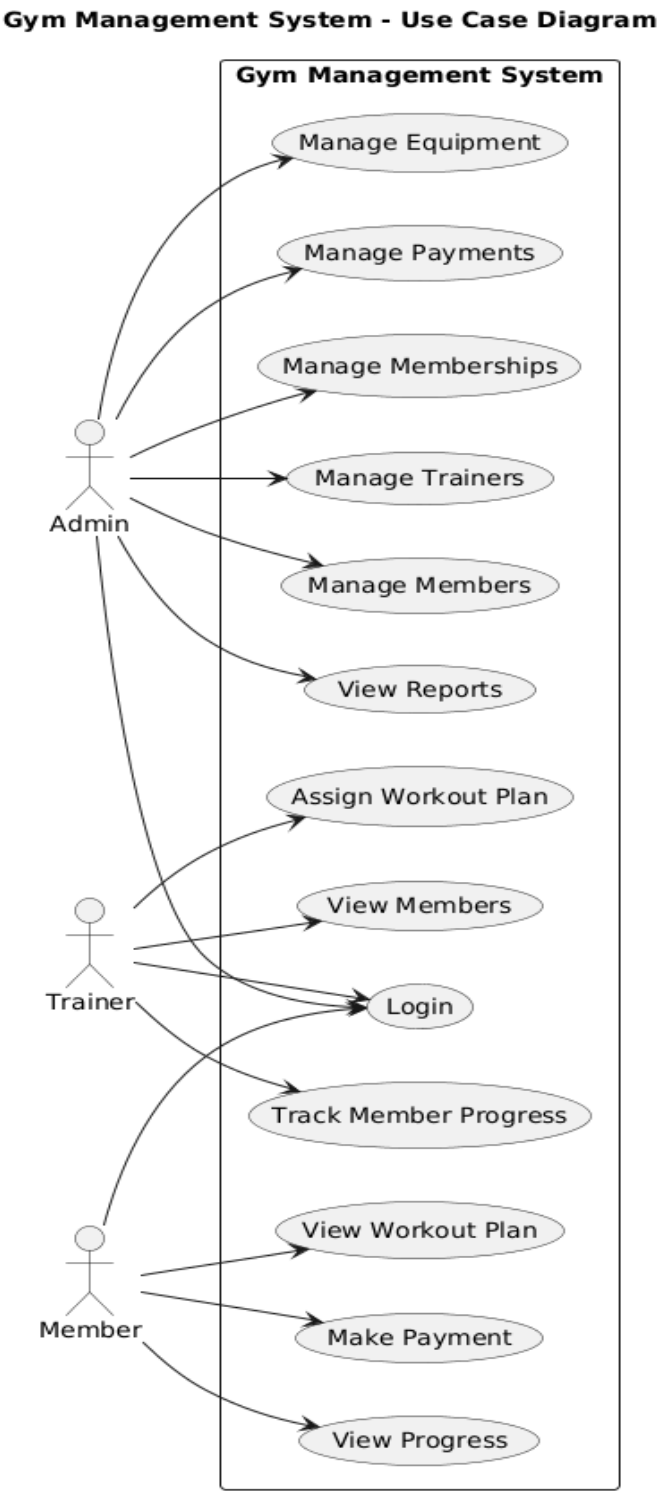


Fig. 3.2 Use Case Diagram of GYM Management System.

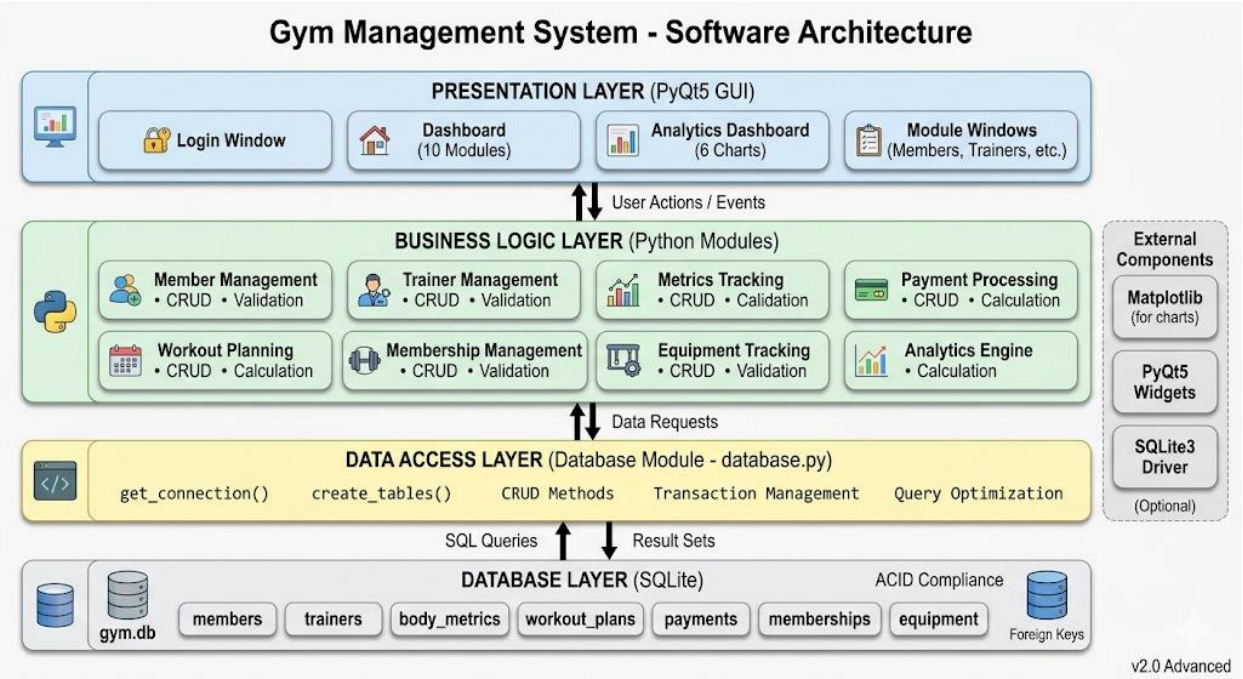


Fig. 3.3 Architecture Diagram of GYM Management System.

3.2 Front-End Design:

The front-end design of the Gym Management System focuses on creating an intuitive and user-friendly interface that allows users to interact with the system efficiently. The interface is developed using the PyQt5 framework, which provides a wide range of tools for designing modern graphical user interfaces.

User Interface Structure

The application consists of multiple windows and screens, each designed for a specific purpose:

- **Login Window**
  The login interface provides secure access to the system. It includes input fields for username and password, along with validation mechanisms to ensure authorized access.
- **Dashboard**
  The dashboard acts as the main control panel of the system. It provides quick access to all modules through interactive buttons or cards. It also displays key information and navigation options.
- **Module Windows**
  Each module, such as members, trainers, payments, and equipment, has its own interface. These windows allow users to perform operations such as adding, updating, deleting, and viewing records.
- **Analytics Dashboard**
  This interface displays graphical representations of data using charts and graphs. It helps users understand trends and make better decisions.

Design Principles

The front-end design follows several important principles to ensure usability and efficiency:

- **Simplicity**: The interface is designed to be simple and easy to understand, even for non-technical users.
- **Consistency**: Similar elements such as buttons, forms, and layouts are used across all modules for a uniform experience.
- **Responsiveness**: The interface responds quickly to user actions, providing immediate feedback.
- **Clarity**: Labels, buttons, and instructions are clearly defined to avoid confusion.

User Interaction Flow

The system follows a structured user interaction flow. After logging in, the user is directed to the dashboard, from where they can navigate to different modules. Within each module, users can perform operations such as adding new records or viewing existing data. Once an action is completed, the system provides confirmation messages or updates the display accordingly.

GUI Components Used

The front-end design uses various PyQt5 components, including:

- Text fields for input
- Buttons for actions
- Tables for displaying data
- Dropdown menus for selection
- Dialog boxes for notifications

These components are arranged using layout managers to ensure proper alignment and spacing.

Visualization and Feedback

The system includes basic data visualization features using charts and graphs, which help users understand information more effectively. Feedback mechanisms such as success messages, error alerts, and confirmation dialogs improve user experience and ensure smooth operation.

Advantages of Front-End Design

A well-designed front end enhances usability and makes the system more accessible to users. It reduces the learning curve and increases efficiency by providing clear navigation and easy interaction. The use of PyQt5 allows for the creation of visually appealing and responsive interfaces, making the application more professional.

Conclusion of Design

The design of the Gym Management System ensures a well-structured and efficient application by combining a strong database design with an intuitive user interface. The E-R design provides a solid foundation for data management, while the front-end design ensures ease of use and accessibility. Together, they form a complete and effective system capable of handling modern gym management requirements.

CHAPTER 4

PROPOSED METHODOLOGY

4.1 Modules Description:

The Gym Management System is divided into several modules, each responsible for handling a specific part of the system:

- **Member Management Module**
  This module handles the registration, updating, and deletion of member details. It stores information such as personal details, contact information, membership type, and joining date. It ensures proper validation and easy retrieval of member records.
- **Trainer Management Module**
  This module manages trainer information, including their profiles, schedules, and assigned members. It helps in organizing trainer activities and ensures efficient allocation of resources.
- **Membership Management Module**
  This module handles different membership plans, pricing, and duration. It also tracks membership validity and expiry dates, helping staff manage renewals effectively.
- **Payment Management Module**
  This module records and tracks all payment transactions made by members. It maintains payment history, supports billing operations, and helps in identifying pending dues.
- **Workout Plan Module**
  This module allows the creation and assignment of workout plans based on individual member requirements. It helps trainers track progress and update workout routines accordingly.
- **Equipment Management Module**
  This module manages gym equipment details, including inventory, condition, and maintenance schedules. It ensures proper utilization and timely maintenance of equipment.
- **Metrics and Progress Tracking Module**
  This module records fitness-related data such as weight, BMI, and other performance metrics. It helps in tracking member progress over time.
- **Analytics and Reporting Module**
  This module generates basic reports and visual insights related to membership growth, revenue, and overall gym performance. It supports better decision-making.

Each module is designed to work independently while maintaining integration with other modules, ensuring smooth and efficient system operation.

4.2 Database Connectivity:

The Gym Management System uses **SQLite** as its database management system to store and manage data efficiently. Database connectivity is handled through Python using the built-in **sqlite3** library, which allows seamless interaction between the application and the database.

A dedicated database file (gym.db) is used to store all data related to members, trainers, payments, workout plans, and equipment. The system establishes a connection to the database using a centralized connection method, ensuring consistency and efficient resource management.

The application performs various database operations such as inserting new records, updating existing data, deleting unnecessary entries, and retrieving required information using structured SQL queries. Prepared statements and validation techniques are used to ensure data security and prevent errors.

Transaction management is implemented to maintain data integrity, with operations such as commit and rollback ensuring that changes are saved correctly or reverted in case of failure. The database is designed following normalization principles to reduce redundancy and improve efficiency.

Overall, the database connectivity in the proposed system ensures fast, reliable, and secure data handling, enabling smooth interaction between the user interface and the backend database.

CHAPTER 5

IMPLEMENTATION

The implementation of the Gym Management System focuses on developing a reliable, efficient, and user-friendly desktop-based application by integrating a well-structured backend with an interactive and responsive frontend. The system is developed using Python as the primary programming language for handling application logic, while SQLite is used as the database management system for storing and managing data. This combination provides a lightweight yet powerful solution suitable for small to medium-scale applications. The overall implementation follows a modular and layered approach, where each functionality is designed as an independent module and later integrated into a unified system. This modular design improves maintainability, scalability, and simplifies debugging and future enhancements.

The system is designed to handle all essential gym operations in an organized and systematic manner. These operations include member management, trainer management, membership handling, payment tracking, workout plan assignment, and equipment management. Each of these functionalities is carefully implemented to ensure smooth interaction between different parts of the system. The frontend of the application provides a graphical user interface that allows users to easily interact with the system without requiring technical knowledge. The interface includes various forms, buttons, and dashboards that guide the user through different operations such as adding new members, updating records, and viewing reports.

On the other hand, the backend is responsible for managing data storage, processing, and retrieval. It ensures that all data entered by the user is securely stored in the database and can be accessed whenever required. The backend logic is implemented using Python functions that interact with the SQLite database through structured queries. These functions handle operations such as inserting new records, updating existing information, deleting unnecessary data, and retrieving relevant details based on user requests. Proper validation mechanisms are incorporated to ensure that only correct and meaningful data is stored in the system, thereby reducing errors and maintaining data integrity.

Communication between the frontend and backend is achieved through well-defined function calls. When a user performs an action in the interface, such as clicking a button or submitting a form, the corresponding Python function is triggered. This function processes the input, executes the required SQL query, and returns the result to the frontend, which is then displayed to the user. This seamless interaction ensures a smooth and efficient workflow within the application.

In addition, the implementation emphasizes data consistency and reliability. Transaction management techniques are used to ensure that all database operations are executed correctly. In case of any failure during a transaction, appropriate measures are taken to prevent data corruption. The database is designed using proper structure and relationships to minimize redundancy and improve performance. Indexing and optimized queries are used to enhance the speed of data retrieval, making the system responsive even when handling large amounts of data.

The frontend design also plays a crucial role in the implementation. Using the PyQt5 framework, the application provides a visually appealing and easy-to-use interface. The layout is designed to be simple and intuitive, allowing users to navigate through different modules with ease. Consistent design elements, proper alignment, and clear labeling of buttons and fields improve the overall user experience. Additional features such as confirmation messages, alerts, and basic data visualization further enhance usability.

Overall, the implementation of the Gym Management System successfully combines frontend and backend technologies to create a complete and functional application. It replaces manual processes with an automated system, improving efficiency, accuracy, and reliability. The modular structure and clear separation of components ensure that the system can be easily maintained and extended in the future. This makes the application a practical and effective solution for modern gym management needs.

5.1 Back end via SQLite and Python:

database.py

```
import sqlite3
from datetime import datetime
class Database:
    def __init__(self, db_name='gym.db'):
        self.db_name = db_name
        self.create_tables()
        self.insert_default_data()
    def get_connection(self):
        return sqlite3.connect(self.db_name)
    def create_tables(self):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS memberships (
                membership_id INTEGER PRIMARY KEY AUTOINCREMENT,
                type TEXT NOT NULL,
                fee REAL NOT NULL,
                duration INTEGER NOT NULL
            )
        ''')
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS trainers (
                trainer_id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                specialization TEXT,
                phone TEXT,
                email TEXT,
                salary REAL
            )
        ''')
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS members (
                member_id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                age INTEGER,
                gender TEXT,
                phone TEXT,
                email TEXT,
                address TEXT,
```

```
            join_date TEXT,
            membership_id INTEGER,
            trainer_id INTEGER,
            FOREIGN KEY (membership_id) REFERENCES
memberships(membership_id),
            FOREIGN KEY (trainer_id) REFERENCES trainers(trainer_id)
        )
    ''')
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS body_metrics (
            record_id INTEGER PRIMARY KEY AUTOINCREMENT,
            member_id INTEGER,
            date TEXT,
            weight REAL,
            height REAL,
            bmi REAL,
            FOREIGN KEY (member_id) REFERENCES members(member_id)
        )
    ''')
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS workout_plans (
            plan_id INTEGER PRIMARY KEY AUTOINCREMENT,
            member_id INTEGER,
            trainer_id INTEGER,
            goal TEXT,
            level TEXT,
            duration INTEGER,
            FOREIGN KEY (member_id) REFERENCES members(member_id),
            FOREIGN KEY (trainer_id) REFERENCES trainers(trainer_id)
        )
    ''')
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS payments (
            payment_id INTEGER PRIMARY KEY AUTOINCREMENT,
            member_id INTEGER,
            date TEXT,
            amount REAL,
            payment_mode TEXT,
            FOREIGN KEY (member_id) REFERENCES members(member_id)
        )
    ''')
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS equipment (
```

```
            equipment_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            type TEXT,
            purchase_date TEXT,
            condition TEXT
        )
    ''')

    conn.commit()
    conn.close()
  def insert_default_data(self):
    conn = self.get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT COUNT(*) FROM memberships")
    if cursor.fetchone()[0] == 0:
        default_plans = [
            ('Monthly', 1500, 30),
            ('Quarterly', 4000, 90),
            ('Half-Yearly', 7500, 180),
            ('Yearly', 14000, 365)
        ]
        cursor.executemany(
            "INSERT INTO memberships (type, fee, duration) VALUES (?, ?, ?)",
            default_plans
        )
        conn.commit()
    conn.close()
  def add_member(self, name, age, gender, phone, email, address, membership_id,
trainer_id):
    conn = self.get_connection()
    cursor = conn.cursor()
    join_date = datetime.now().strftime('%Y-%m-%d')
    cursor.execute('''
        INSERT INTO members (name, age, gender, phone, email, address, join_date,
membership_id, trainer_id)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    ''', (name, age, gender, phone, email, address, join_date, membership_id,
trainer_id))
    conn.commit()
    conn.close()
  def get_all_members(self):
    conn = self.get_connection()
    cursor = conn.cursor()
```

```
        cursor.execute('''
            SELECT m.member_id, m.name, m.age, m.gender, m.phone, m.email,
                   m.address, m.join_date,
                   mb.type as membership, t.name as trainer
            FROM members m
            LEFT JOIN memberships mb ON m.membership_id = mb.membership_id
            LEFT JOIN trainers t ON m.trainer_id = t.trainer_id
        ''')
        members = cursor.fetchall()
        conn.close()
        return members

    def update_member(self, member_id, name, age, gender, phone, email, address,
membership_id, trainer_id):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            UPDATE members
            SET name=?, age=?, gender=?, phone=?, email=?, address=?,
membership_id=?, trainer_id=?
            WHERE member_id=?
        ''', (name, age, gender, phone, email, address, membership_id, trainer_id,
member_id))
        conn.commit()
        conn.close()
    def delete_member(self, member_id):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute("DELETE FROM members WHERE member_id=?",
(member_id,))
        conn.commit()
        conn.close()
    def add_trainer(self, name, specialization, phone, email, salary):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO trainers (name, specialization, phone, email, salary)
            VALUES (?, ?, ?, ?, ?)
        ''', (name, specialization, phone, email, salary))
        conn.commit()
        conn.close()
    def get_all_trainers(self):
        conn = self.get_connection()
```

```
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM trainers")
    trainers = cursor.fetchall()
    conn.close()
    return trainers
def update_trainer(self, trainer_id, name, specialization, phone, email, salary):
    conn = self.get_connection()
    cursor = conn.cursor()
    cursor.execute('''
        UPDATE trainers
        SET name=?, specialization=?, phone=?, email=?, salary=?
        WHERE trainer_id=?
    ''', (name, specialization, phone, email, salary, trainer_id))
    conn.commit()
    conn.close()

def delete_trainer(self, trainer_id):
    conn = self.get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM trainers WHERE trainer_id=?", (trainer_id,))
    conn.commit()
    conn.close()
def add_body_metric(self, member_id, weight, height):
    conn = self.get_connection()
    cursor = conn.cursor()
    date = datetime.now().strftime('%Y-%m-%d')
    height_m = height / 100
    bmi = weight / (height_m ** 2)
    cursor.execute('''
        INSERT INTO body_metrics (member_id, date, weight, height, bmi)
        VALUES (?, ?, ?, ?, ?)
    ''', (member_id, date, weight, height, round(bmi, 2)))
    conn.commit()
    conn.close()
def get_member_metrics(self, member_id):
    conn = self.get_connection()
    cursor = conn.cursor()
    cursor.execute('''
        SELECT record_id, date, weight, height, bmi
        FROM body_metrics
        WHERE member_id=?
        ORDER BY date DESC
    ''', (member_id,))
```

```
        metrics = cursor.fetchall()
        conn.close()
        return metrics
    def add_workout_plan(self, member_id, trainer_id, goal, level, duration):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO workout_plans (member_id, trainer_id, goal, level, duration)
            VALUES (?, ?, ?, ?, ?)
        ''', (member_id, trainer_id, goal, level, duration))
        conn.commit()
        conn.close()
    def get_all_workout_plans(self):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            SELECT wp.plan_id, m.name as member, t.name as trainer,
                   wp.goal, wp.level, wp.duration
            FROM workout_plans wp
            JOIN members m ON wp.member_id = m.member_id
            JOIN trainers t ON wp.trainer_id = t.trainer_id
        ''')
        plans = cursor.fetchall()
        conn.close()
        return plans
    def add_payment(self, member_id, amount, payment_mode):
        conn = self.get_connection()
        cursor = conn.cursor()
        date = datetime.now().strftime('%Y-%m-%d')
        cursor.execute('''
            INSERT INTO payments (member_id, date, amount, payment_mode)
            VALUES (?, ?, ?, ?)
        ''', (member_id, date, amount, payment_mode))
        conn.commit()
        conn.close()
    def get_member_payments(self, member_id):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            SELECT payment_id, date, amount, payment_mode
            FROM payments
            WHERE member_id=?
            ORDER BY date DESC
```

```
        ''', (member_id,))
        payments = cursor.fetchall()
        conn.close()
        return payments
    def get_all_payments(self):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            SELECT p.payment_id, m.name as member, p.date, p.amount,
p.payment_mode
            FROM payments p
            JOIN members m ON p.member_id = m.member_id
            ORDER BY p.date DESC
        ''')
        payments = cursor.fetchall()
        conn.close()
        return payments
    def add_membership(self, type, fee, duration):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO memberships (type, fee, duration)
            VALUES (?, ?, ?)
        ''', (type, fee, duration))
        conn.commit()
        conn.close()
    def get_all_memberships(self):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM memberships")
        memberships = cursor.fetchall()
        conn.close()
        return memberships
    def update_membership(self, membership_id, type, fee, duration):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            UPDATE memberships
            SET type=?, fee=?, duration=?
            WHERE membership_id=?
        ''', (type, fee, duration, membership_id))
        conn.commit()
        conn.close()
```

```
    def delete_membership(self, membership_id):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute("DELETE FROM memberships WHERE membership_id=?",
(membership_id,))
        conn.commit()
        conn.close()
    def add_equipment(self, name, type, purchase_date, condition):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO equipment (name, type, purchase_date, condition)
            VALUES (?, ?, ?, ?)
        ''', (name, type, purchase_date, condition))
        conn.commit()
        conn.close()
    def get_all_equipment(self):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM equipment")
        equipment = cursor.fetchall()
        conn.close()
        return equipment
def update_equipment(self, equipment_id, name, type, purchase_date, condition):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute('''
            UPDATE equipment
            SET name=?, type=?, purchase_date=?, condition=?
            WHERE equipment_id=?
        ''', (name, type, purchase_date, condition, equipment_id))
        conn.commit()
        conn.close()
    def delete_equipment(self, equipment_id):
        conn = self.get_connection()
        cursor = conn.cursor()
        cursor.execute("DELETE FROM equipment WHERE equipment_id=?",
(equipment_id,))
        conn.commit()
        conn.close()
```

5.2 Front End Via Python:

main.py

```
import sys
from PyQt5.QtWidgets import QApplication
from login import LoginWindow
from dashboard import Dashboard
class GymManagementSystem:
    def __init__(self):
        self.app = QApplication(sys.argv)
        self.app.setApplicationName("Gym Management System")
        self.app.setStyle('Fusion')
        self.login_window = LoginWindow()
        self.login_window.login_successful.connect(self.show_dashboard)
        self.login_window.show()
    def show_dashboard(self):
        self.dashboard = Dashboard()
        self.dashboard.show()
    def run(self):
        sys.exit(self.app.exec_())
if __name__ == '__main__':
    gym_app = GymManagementSystem()
    gym_app.run()
```

login.py

```
from PyQt5.QtWidgets import (QWidget, QLabel, QLineEdit, QPushButton,
                             QVBoxLayout, QHBoxLayout, QFrame,
QGraphicsDropShadowEffect)
from PyQt5.QtCore import Qt, pyqtSignal, QPropertyAnimation, QPoint, QTimer
from PyQt5.QtGui import QFont, QColor
class LoginWindow(QWidget):
    login_successful = pyqtSignal()
    def __init__(self):
        super().__init__()
        self.init_ui()
    def init_ui(self):
        self.setWindowTitle("Gym Management System - Login")
        self.setGeometry(100, 100, 500, 650)
        self.setFixedSize(500, 650)
        self.setObjectName("MainWindow")
        self.setStyleSheet("""
            QWidget#MainWindow {
                background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
```

```
                stop:0 #0f2027, stop:0.5 #203a43, stop:1 #2c5364);
            }
        """)
        layout = QVBoxLayout()
        layout.setAlignment(Qt.AlignCenter)
        self.login_card = QFrame()
        self.login_card.setStyleSheet("""
            QFrame {
                background: rgba(20, 25, 30, 0.7);
                border-radius: 20px;
                border: 1px solid rgba(255, 255, 255, 0.08);
            }
        """)
        self.login_card.setFixedSize(420, 520)
        shadow = QGraphicsDropShadowEffect()
        shadow.setBlurRadius(50)
        shadow.setColor(QColor(0, 0, 0, 150))
        shadow.setOffset(0, 20)
        self.login_card.setGraphicsEffect(shadow)
        card_layout = QVBoxLayout()
        card_layout.setContentsMargins(40, 40, 40, 40)
        card_layout.setSpacing(20)
        logo = QLabel("🏋")
        logo.setFont(QFont("Segoe UI", 40))
        logo.setAlignment(Qt.AlignCenter)
        logo.setStyleSheet("background: transparent; border: none; color: #00d2ff;")
        card_layout.addWidget(logo)
        title = QLabel("GYM Management System")
        title.setFont(QFont("Segoe UI Black", 15))
        title.setStyleSheet("color: white; background: transparent; border: none; letter-
spacing: 2px;")
        title.setAlignment(Qt.AlignCenter)
        card_layout.addWidget(title)
        subtitle = QLabel("Welcome Back!")
        subtitle.setFont(QFont("Segoe UI", 11, QFont.Bold))
        subtitle.setStyleSheet("color: #00d2ff; background: transparent; border: none;
letter-spacing: 4px;")
        subtitle.setAlignment(Qt.AlignCenter)
        card_layout.addWidget(subtitle)
        card_layout.addSpacing(15)
        username_container = QFrame()
        username_container.setStyleSheet("""
            QFrame {
```

```
            background: rgba(0, 0, 0, 0.4);
            border-radius: 12px;
            border: 1px solid rgba(255, 255, 255, 0.05);
        }
        QFrame:hover {
            border: 1px solid rgba(0, 210, 255, 0.5);
        }
    """)
    username_layout = QHBoxLayout()
    username_layout.setContentsMargins(15, 8, 15, 8)
    username_icon = QLabel("👤")
    username_icon.setFont(QFont("Segoe UI", 16))
    username_icon.setStyleSheet("background: transparent; border: none; color:
#a0a5a8;")
    self.username_input = QLineEdit()
    self.username_input.setPlaceholderText("Username")
    self.username_input.setStyleSheet("""
        QLineEdit {
            background: transparent;
            border: none;
            color: white;
            font-family: 'Segoe UI';
            font-size: 15px;
            padding: 5px;
        }
        QLineEdit::placeholder {
            color: #6c757d;
        }
    """)
    username_layout.addWidget(username_icon)
    username_layout.addWidget(self.username_input)
    username_container.setLayout(username_layout)
    card_layout.addWidget(username_container)
    password_container = QFrame()
    password_container.setStyleSheet("""
        QFrame {
            background: rgba(0, 0, 0, 0.4);
            border-radius: 12px;
            border: 1px solid rgba(255, 255, 255, 0.05);
        }
        QFrame:hover {
            border: 1px solid rgba(0, 210, 255, 0.5);
        }
```

```
        """)
        password_layout = QHBoxLayout()
        password_layout.setContentsMargins(15, 8, 15, 8)
        password_icon = QLabel("🔒")
        password_icon.setFont(QFont("Segoe UI", 16))
        password_icon.setStyleSheet("background: transparent; border: none; color:
#a0a5a8;")
        self.password_input = QLineEdit()
        self.password_input.setPlaceholderText("Password")
        self.password_input.setEchoMode(QLineEdit.Password)
        self.password_input.setStyleSheet("""
            QLineEdit {
                background: transparent;
                border: none;
                color: white;
                font-family: 'Segoe UI';
                font-size: 15px;
                padding: 5px;
            }
            QLineEdit::placeholder {
                color: #6c757d;
            }
        """)
        password_layout.addWidget(password_icon)
        password_layout.addWidget(self.password_input)
        password_container.setLayout(password_layout)
        card_layout.addWidget(password_container)
        card_layout.addSpacing(15)
        self.login_btn = QPushButton("SECURE LOGIN")
        self.login_btn.setFont(QFont("Segoe UI", 13, QFont.Bold))
        self.login_btn.setCursor(Qt.PointingHandCursor)
        self.login_btn.setStyleSheet("""
            QPushButton {
                background: qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 #0052D4,
stop:0.5 #4364F7, stop:1 #6FB1FC);
                color: white;
                border: none;
                border-radius: 12px;
                padding: 16px;
                letter-spacing: 1px;
            }
            QPushButton:hover {
                background: qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 #6FB1FC,
```

```
stop:0.5 #4364F7, stop:1 #0052D4);
            }
            QPushButton:pressed {
                background: #003a99;
            }
        """)
        self.login_btn.clicked.connect(self.check_login)
        card_layout.addWidget(self.login_btn)

        info_label = QLabel("Username: admin / Password: admin")
        info_label.setFont(QFont("Segoe UI", 9))
        info_label.setStyleSheet("color: #6c757d; background: transparent; border:
none;")
        info_label.setAlignment(Qt.AlignCenter)
        card_layout.addWidget(info_label)
        self.login_card.setLayout(card_layout)
        layout.addWidget(self.login_card)
        self.setLayout(layout)
        self.password_input.returnPressed.connect(self.check_login)
    def check_login(self):
        username = self.username_input.text()
        password = self.password_input.text()
        if username == "admin" and password == "admin":
            self.login_btn.setText("✓ ACCESS GRANTED")
            self.login_btn.setStyleSheet("""
                QPushButton {
                    background: qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 #11998e,
stop:1 #38ef7d);
                    color: white;
                    border: none;
                    border-radius: 12px;
                    padding: 16px;
                    font-weight: bold;
                    letter-spacing: 1px;
                }
            """)
            QTimer.singleShot(500, self.login_successful.emit)
            QTimer.singleShot(500, self.close)
        else:
            self.shake_window()
            self.login_btn.setText("✗ ACCESS DENIED")
            self.login_btn.setStyleSheet("""
                QPushButton {
```

```
                background: qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 #cb2d3e,
stop:1 #ef473a);
                color: white;
                border: none;
                border-radius: 12px;
                padding: 16px;
                font-weight: bold;
                letter-spacing: 1px;
            }
        """)
        QTimer.singleShot(2000, self.reset_login_button)
        self.password_input.clear()
        self.password_input.setFocus()
    def shake_window(self):
        original_pos = self.pos()
        self.shake_anim = QPropertyAnimation(self, b"pos")
        self.shake_anim.setDuration(400)
        self.shake_anim.setLoopCount(1)
        self.shake_anim.setKeyValueAt(0, original_pos)
        self.shake_anim.setKeyValueAt(0.1, original_pos + QPoint(-10, 0))
        self.shake_anim.setKeyValueAt(0.2, original_pos + QPoint(10, 0))
        self.shake_anim.setKeyValueAt(0.3, original_pos + QPoint(-10, 0))
        self.shake_anim.setKeyValueAt(0.4, original_pos + QPoint(10, 0))
        self.shake_anim.setKeyValueAt(0.5, original_pos + QPoint(-10, 0))
        self.shake_anim.setKeyValueAt(0.6, original_pos + QPoint(10, 0))
        self.shake_anim.setKeyValueAt(0.7, original_pos + QPoint(-5, 0))
        self.shake_anim.setKeyValueAt(0.8, original_pos + QPoint(5, 0))
        self.shake_anim.setKeyValueAt(0.9, original_pos + QPoint(-2, 0))
        self.shake_anim.setKeyValueAt(1, original_pos)
        self.shake_anim.start()
    def reset_login_button(self):
        self.login_btn.setText("SECURE LOGIN")
        self.login_btn.setStyleSheet("""
            QPushButton {
                background: qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 #0052D4,
stop:0.5 #4364F7, stop:1 #6FB1FC);
                color: white;
                border: none;
                border-radius: 12px;
                padding: 16px;
                letter-spacing: 1px;
            }
            QPushButton:hover {
```

```
            background: qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 #6FB1FC,
stop:0.5 #4364F7, stop:1 #0052D4);
        }
        QPushButton:pressed {
            background: #003a99;
        }
    """)
```

dashboard.py

```
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel, QPushButton,
QVBoxLayout,
                             QHBoxLayout, QGridLayout, QFrame,
QGraphicsDropShadowEffect, QScrollArea)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont, QColor
class DashboardCard(QPushButton):
    def __init__(self, title, subtitle, icon_text, neon_color, parent=None):
        super().__init__(parent)
        self.setCursor(Qt.PointingHandCursor)
        self.setFixedHeight(150)
        self.neon_color = neon_color
        layout = QVBoxLayout(self)
        layout.setContentsMargins(25, 25, 25, 25)
        layout.setSpacing(8)
        self.icon_label = QLabel(icon_text)
        self.icon_label.setFont(QFont("Segoe UI Emoji", 32))
        self.icon_label.setStyleSheet("background: transparent; border: none;")
        self.icon_label.setAlignment(Qt.AlignLeft)
        self.title_label = QLabel(title)
        self.title_label.setFont(QFont("Segoe UI", 14, QFont.Bold))
        self.title_label.setStyleSheet("color: white; background: transparent; border:
none;")
        self.title_label.setAlignment(Qt.AlignLeft)
        self.subtitle_label = QLabel(subtitle)
        self.subtitle_label.setFont(QFont("Segoe UI", 10))
        self.subtitle_label.setStyleSheet("color: #a0a5a8; background: transparent;
border: none;")
        self.subtitle_label.setAlignment(Qt.AlignLeft)
        self.subtitle_label.setWordWrap(True)
        layout.addWidget(self.icon_label)
        layout.addWidget(self.title_label)
        layout.addWidget(self.subtitle_label)
```

```
        layout.addStretch()
        self.setStyleSheet(f"""
            QPushButton {{
                background-color: rgba(0, 0, 0, 0.4);
                border-radius: 16px;
                border: 1px solid rgba(255, 255, 255, 0.05);
            }}
            QPushButton:hover {{
                background-color: rgba(20, 30, 40, 0.8);
                border: 1px solid {self.neon_color};
            }}
            QPushButton:pressed {{
                background-color: rgba(0, 0, 0, 0.8);
                border: 1px solid {self.neon_color};
            }}
        """)
        shadow = QGraphicsDropShadowEffect()
        shadow.setBlurRadius(25)
        shadow.setXOffset(0)
        shadow.setYOffset(8)
        shadow.setColor(QColor(0, 0, 0, 100))
        self.setGraphicsEffect(shadow)
class Dashboard(QWidget):
    def __init__(self):
        super().__init__()
        self.init_ui()
    def init_ui(self):
        self.setWindowTitle("Gym Management System - Dashboard")
        self.setMinimumSize(900, 600)
        self.resize(1100, 750)
        self.setObjectName("DashboardMain")
        self.setStyleSheet("""
            QWidget#DashboardMain {
                background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
                    stop:0 #0f2027, stop:0.5 #203a43, stop:1 #2c5364);
            }
        """)
        main_layout = QHBoxLayout(self)
        main_layout.setContentsMargins(0, 0, 0, 0)
        main_layout.setSpacing(0)
        sidebar = QFrame()
        sidebar.setFixedWidth(280)
        sidebar.setStyleSheet("""
```

```
            QFrame {
                background-color: rgba(10, 15, 20, 0.7);
                border-right: 1px solid rgba(255, 255, 255, 0.08);
            }
        """)
        sidebar_layout = QVBoxLayout(sidebar)
        sidebar_layout.setContentsMargins(20, 40, 20, 20)
        brand = QLabel("GYM")
        brand.setFont(QFont("Segoe UI", 22, QFont.Bold))
        brand.setStyleSheet("color: white; background: transparent; border: none;
letter-spacing: 2px;")
        brand.setAlignment(Qt.AlignCenter)
        sidebar_layout.addWidget(brand)
        sidebar_layout.addSpacing(40)
        welcome = QLabel("Welcome Admin!")
        welcome.setFont(QFont("Segoe UI", 11))
        welcome.setStyleSheet("color: #00d2ff; background: transparent; border: none;
letter-spacing: 1px;")
        welcome.setAlignment(Qt.AlignCenter)
        sidebar_layout.addWidget(welcome)
        sidebar_layout.addStretch()
        footer = QLabel("GYM Management System")
        footer.setFont(QFont("Segoe UI", 9, QFont.Bold))
        footer.setStyleSheet("color: #6c757d; background: transparent; border: none;
letter-spacing: 1px;")
        footer.setAlignment(Qt.AlignCenter)
        sidebar_layout.addWidget(footer)
        main_layout.addWidget(sidebar)
        scroll_area = QScrollArea()
        scroll_area.setWidgetResizable(True)
        scroll_area.setFrameShape(QFrame.NoFrame)
        scroll_area.setStyleSheet("""
            QScrollArea {
                background: transparent;
                border: none;
            }
            QScrollBar:vertical {
                border: none;
                background: rgba(255, 255, 255, 0.05);
                width: 10px;
                border-radius: 5px;
            }
            QScrollBar::handle:vertical {
```

```
            background: rgba(255, 255, 255, 0.2);
            border-radius: 5px;
        }
        QScrollBar::handle:vertical:hover {
            background: rgba(0, 210, 255, 0.5);
        }
        QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {
            height: 0px;
        }
    """)
    content_area = QWidget()
    content_area.setObjectName("ScrollContent")
    content_area.setStyleSheet("QWidget#ScrollContent { background:
transparent; }")
    content_layout = QVBoxLayout(content_area)
    content_layout.setContentsMargins(40, 40, 40, 40)
    header_text = QLabel("Management Center")
    header_text.setFont(QFont("Segoe UI Black", 26))
    header_text.setStyleSheet("color: white; background: transparent; letter-
spacing: 2px;")
    content_layout.addWidget(header_text)
    tagline = QLabel("Real-Time Database Analytics")
    tagline.setFont(QFont("Segoe UI", 11))
    tagline.setStyleSheet("color: #a0a5a8; background: transparent;")
    content_layout.addWidget(tagline)
    content_layout.addSpacing(30)
    grid_layout = QGridLayout()
    grid_layout.setSpacing(25)
    for i in range(3):
        grid_layout.setColumnStretch(i, 1)
    buttons_data = [
        ("Members", "Manage gym members", "👥", "#00d2ff",
self.open_members),
        ("Trainers", "Manage training staff", "🏋", "#38ef7d", self.open_trainers),
        ("Body Metrics", "Track client progress", "⚖", "#f64f59",
self.open_metrics),
        ("Workout Plans", "Create custom routines", "📋", "#c471ed",
self.open_plans),
        ("Payments", "Record transactions", "💳", "#fbd786", self.open_payments),
        ("Memberships", "Manage plan tiers", "🏷", "#11998e",
self.open_memberships),
        ("Equipment", "Track gym inventory", "🏋", "#f7797d",
```

```
self.open_equipment),
            ("Analytics", "View business insights", "📊", "#00c6ff",
self.open_analytics),
            ("Exit System", "Securely close app", "🚪", "#eb3349", self.close)
        ]
        row, col = 0, 0
        for text, sub, icon, neon_color, func in buttons_data:
            card = DashboardCard(text, sub, icon, neon_color)
            card.clicked.connect(func)
            grid_layout.addWidget(card, row, col)
            col += 1
            if col > 2:
                col = 0
                row += 1
        content_layout.addLayout(grid_layout)
        content_layout.addStretch()
        scroll_area.setWidget(content_area)
        main_layout.addWidget(scroll_area)
    def open_members(self):
        from members import MembersWindow
        self.members_window = MembersWindow()
        self.members_window.show()
    def open_trainers(self):
        from trainers import TrainersWindow
        self.trainers_window = TrainersWindow()
        self.trainers_window.show()
    def open_metrics(self):
        from metrics import MetricsWindow
        self.metrics_window = MetricsWindow()
        self.metrics_window.show()
    def open_plans(self):
        from plans import WorkoutPlansWindow
        self.plans_window = WorkoutPlansWindow()
        self.plans_window.show()
    def open_payments(self):
        from payments import PaymentsWindow
        self.payments_window = PaymentsWindow()
        self.payments_window.show()
    def open_memberships(self):
        from memberships import MembershipsWindow
        self.memberships_window = MembershipsWindow()
        self.memberships_window.show()
    def open_equipment(self):
```

```
        from equipment import EquipmentWindow
        self.equipment_window = EquipmentWindow()
        self.equipment_window.show()
    def open_analytics(self):
        from analytics import AnalyticsWindow
        self.analytics_window = AnalyticsWindow()
        self.analytics_window.show()
if __name__ == '__main__':
    app = QApplication(sys.argv)
    font = QFont("Segoe UI", 10)
    app.setFont(font)
    dashboard = Dashboard()
    dashboard.show()
    sys.exit(app.exec_())
```

members.py

```
from PyQt5.QtWidgets import (QWidget, QLabel, QLineEdit, QPushButton,
QTableWidget,
                             QTableWidgetItem, QVBoxLayout, QHBoxLayout,
QGridLayout,
                             QComboBox, QMessageBox, QHeaderView)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from database import Database
class MembersWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.db = Database()
        self.init_ui()
        self.load_members()
        self.load_memberships()
        self.load_trainers()
    def init_ui(self):
        self.setWindowTitle("Members Management")
        self.setGeometry(100, 100, 1200, 700)
        main_layout = QVBoxLayout()
        header = QLabel("Members Management")
        header.setFont(QFont("Arial", 18, QFont.Bold))
        header.setStyleSheet("color: #2c3e50; padding: 10px;")
        header.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(header)
        form_layout = QGridLayout()
        self.member_id_input = QLineEdit()
```

```
        self.member_id_input.setPlaceholderText("Auto")
        self.member_id_input.setReadOnly(True)
        self.name_input = QLineEdit()
        self.name_input.setPlaceholderText("Enter full name")
        self.age_input = QLineEdit()
        self.age_input.setPlaceholderText("Enter age")
        self.gender_combo = QComboBox()
        self.gender_combo.addItems(["Male", "Female", "Other"])
        self.phone_input = QLineEdit()
        self.phone_input.setPlaceholderText("Enter phone number")
        self.email_input = QLineEdit()
        self.email_input.setPlaceholderText("Enter email")
        self.address_input = QLineEdit()
        self.address_input.setPlaceholderText("Enter address")
        self.membership_combo = QComboBox()
        self.trainer_combo = QComboBox()
        form_layout.addWidget(QLabel("Member ID:"), 0, 0)
        form_layout.addWidget(self.member_id_input, 0, 1)
        form_layout.addWidget(QLabel("Name:"), 0, 2)
        form_layout.addWidget(self.name_input, 0, 3)
        form_layout.addWidget(QLabel("Age:"), 1, 0)
        form_layout.addWidget(self.age_input, 1, 1)
        form_layout.addWidget(QLabel("Gender:"), 1, 2)
        form_layout.addWidget(self.gender_combo, 1, 3)
        form_layout.addWidget(QLabel("Phone:"), 2, 0)
        form_layout.addWidget(self.phone_input, 2, 1)
        form_layout.addWidget(QLabel("Email:"), 2, 2)
        form_layout.addWidget(self.email_input, 2, 3)
        form_layout.addWidget(QLabel("Address:"), 3, 0)
        form_layout.addWidget(self.address_input, 3, 1, 1, 3)
        form_layout.addWidget(QLabel("Membership:"), 4, 0)
        form_layout.addWidget(self.membership_combo, 4, 1)
        form_layout.addWidget(QLabel("Trainer:"), 4, 2)
        form_layout.addWidget(self.trainer_combo, 4, 3)
        main_layout.addLayout(form_layout)
        buttons_layout = QHBoxLayout()
        add_btn = QPushButton("Add Member")
        add_btn.setStyleSheet("background-color: #27ae60; color: white; padding:
8px; font-weight: bold;")
        add_btn.clicked.connect(self.add_member)
        buttons_layout.addWidget(add_btn)
        update_btn = QPushButton("Update Member")
        update_btn.setStyleSheet("background-color: #3498db; color: white; padding:
```

```
8px; font-weight: bold;")
        update_btn.clicked.connect(self.update_member)
        buttons_layout.addWidget(update_btn)
        delete_btn = QPushButton("Delete Member")
        delete_btn.setStyleSheet("background-color: #e74c3c; color: white; padding:
8px; font-weight: bold;")
        delete_btn.clicked.connect(self.delete_member)
        buttons_layout.addWidget(delete_btn)
        clear_btn = QPushButton("Clear Form")
        clear_btn.setStyleSheet("background-color: #95a5a6; color: white; padding:
8px; font-weight: bold;")
        clear_btn.clicked.connect(self.clear_form)
        buttons_layout.addWidget(clear_btn)
        main_layout.addLayout(buttons_layout)
        self.table = QTableWidget()
        self.table.setColumnCount(10)
        self.table.setHorizontalHeaderLabels([
            "ID", "Name", "Age", "Gender", "Phone", "Email",
            "Address", "Join Date", "Membership", "Trainer"
        ])
        self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        self.table.cellClicked.connect(self.table_clicked)
        main_layout.addWidget(self.table)
        self.setLayout(main_layout)
    def load_memberships(self):
        self.membership_combo.clear()
        self.membership_combo.addItem("-- Select Membership --", None)
        memberships = self.db.get_all_memberships()
        for membership in memberships:
            self.membership_combo.addItem(
                f"{membership[1]} (₹{membership[2]})",
                membership[0]
            )
    def load_trainers(self):
        self.trainer_combo.clear()
        self.trainer_combo.addItem("-- Select Trainer --", None)
        trainers = self.db.get_all_trainers()
        for trainer in trainers:
            self.trainer_combo.addItem(trainer[1], trainer[0])
    def load_members(self):
        members = self.db.get_all_members()
        self.table.setRowCount(len(members))
```

```
        for row, member in enumerate(members):
            for col, value in enumerate(member):
                item = QTableWidgetItem(str(value) if value else "N/A")
                item.setTextAlignment(Qt.AlignCenter)
                self.table.setItem(row, col, item)

    def add_member(self):
        try:
            name = self.name_input.text().strip()
            age = int(self.age_input.text())
            gender = self.gender_combo.currentText()
            phone = self.phone_input.text().strip()
            email = self.email_input.text().strip()
            address = self.address_input.text().strip()
            membership_id = self.membership_combo.currentData()
            trainer_id = self.trainer_combo.currentData()
            if not name:
                QMessageBox.warning(self, "Error", "Please enter member name!")
                return
            self.db.add_member(name, age, gender, phone, email, address,
membership_id, trainer_id)
            QMessageBox.information(self, "Success", "Member added successfully!")
            self.load_members()
            self.clear_form()
        except ValueError:
            QMessageBox.warning(self, "Error", "Please enter valid age!")
        except Exception as e:
            QMessageBox.critical(self, "Error", f"Failed to add member: {str(e)}")
    def update_member(self):
        try:
            member_id = self.member_id_input.text()
            if not member_id:
                QMessageBox.warning(self, "Error", "Please select a member to update!")
                return
            name = self.name_input.text().strip()
            age = int(self.age_input.text())
            gender = self.gender_combo.currentText()
            phone = self.phone_input.text().strip()
            email = self.email_input.text().strip()
            address = self.address_input.text().strip()
            membership_id = self.membership_combo.currentData()
            trainer_id = self.trainer_combo.currentData()
            self.db.update_member(member_id, name, age, gender, phone, email,
```

```
address,
                        membership_id, trainer_id)
            QMessageBox.information(self, "Success", "Member updated
successfully!")
            self.load_members()
            self.clear_form()
        except ValueError:
            QMessageBox.warning(self, "Error", "Please enter valid age!")
        except Exception as e:
            QMessageBox.critical(self, "Error", f"Failed to update member: {str(e)}")
    def delete_member(self):
        member_id = self.member_id_input.text()
        if not member_id:
            QMessageBox.warning(self, "Error", "Please select a member to delete!")
            return
        reply = QMessageBox.question(self, "Confirm Delete",
                                     "Are you sure you want to delete this member?",
                                     QMessageBox.Yes | QMessageBox.No)
        if reply == QMessageBox.Yes:
            try:
                self.db.delete_member(member_id)
                QMessageBox.information(self, "Success", "Member deleted
successfully!")
                self.load_members()
                self.clear_form()
            except Exception as e:
                QMessageBox.critical(self, "Error", f"Failed to delete member: {str(e)}")
    def table_clicked(self, row):
        self.member_id_input.setText(self.table.item(row, 0).text())
        self.name_input.setText(self.table.item(row, 1).text())
        self.age_input.setText(self.table.item(row, 2).text())
        gender = self.table.item(row, 3).text()
        index = self.gender_combo.findText(gender)
        if index >= 0:
            self.gender_combo.setCurrentIndex(index)
        self.phone_input.setText(self.table.item(row, 4).text())
        self.email_input.setText(self.table.item(row, 5).text())
        self.address_input.setText(self.table.item(row, 6).text())
    def clear_form(self):
        self.member_id_input.clear()
        self.name_input.clear()
        self.age_input.clear()
        self.gender_combo.setCurrentIndex(0)
```

```
        self.phone_input.clear()
        self.email_input.clear()
        self.address_input.clear()
        self.membership_combo.setCurrentIndex(0)
        self.trainer_combo.setCurrentIndex(0)
```

trainers.py

```
from PyQt5.QtWidgets import (QWidget, QLabel, QLineEdit, QPushButton,
QTableWidget,
                             QTableWidgetItem, QVBoxLayout, QHBoxLayout,
QGridLayout,
                             QMessageBox, QHeaderView)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from database import Database
class TrainersWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.db = Database()
        self.init_ui()
        self.load_trainers()
    def init_ui(self):
        self.setWindowTitle("Trainers Management")
        self.setGeometry(100, 100, 1000, 600)
        main_layout = QVBoxLayout()
        header = QLabel("Trainers Management")
        header.setFont(QFont("Arial", 18, QFont.Bold))
        header.setStyleSheet("color: #2c3e50; padding: 10px;")
        header.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(header)
        form_layout = QGridLayout()
        self.trainer_id_input = QLineEdit()
        self.trainer_id_input.setPlaceholderText("Auto")
        self.trainer_id_input.setReadOnly(True)
        self.name_input = QLineEdit()
        self.name_input.setPlaceholderText("Enter trainer name")
        self.specialization_input = QLineEdit()
        self.specialization_input.setPlaceholderText("E.g., Weight Training, Yoga,
Cardio")
        self.phone_input = QLineEdit()
        self.phone_input.setPlaceholderText("Enter phone number")
        self.email_input = QLineEdit()
        self.email_input.setPlaceholderText("Enter email")
```

```
        self.salary_input = QLineEdit()
        self.salary_input.setPlaceholderText("Enter salary amount")
        form_layout.addWidget(QLabel("Trainer ID:"), 0, 0)
        form_layout.addWidget(self.trainer_id_input, 0, 1)
        form_layout.addWidget(QLabel("Name:"), 0, 2)
        form_layout.addWidget(self.name_input, 0, 3)
        form_layout.addWidget(QLabel("Specialization:"), 1, 0)
        form_layout.addWidget(self.specialization_input, 1, 1)
        form_layout.addWidget(QLabel("Phone:"), 1, 2)
        form_layout.addWidget(self.phone_input, 1, 3)
        form_layout.addWidget(QLabel("Email:"), 2, 0)
        form_layout.addWidget(self.email_input, 2, 1)
        form_layout.addWidget(QLabel("Salary (₹):"), 2, 2)
        form_layout.addWidget(self.salary_input, 2, 3)
        main_layout.addLayout(form_layout)
        buttons_layout = QHBoxLayout()
        add_btn = QPushButton("Add Trainer")
        add_btn.setStyleSheet("background-color: #27ae60; color: white; padding:
8px; font-weight: bold;")
        add_btn.clicked.connect(self.add_trainer)
        buttons_layout.addWidget(add_btn)
        update_btn = QPushButton("Update Trainer")
        update_btn.setStyleSheet("background-color: #3498db; color: white; padding:
8px; font-weight: bold;")
        update_btn.clicked.connect(self.update_trainer)
        buttons_layout.addWidget(update_btn)
        delete_btn = QPushButton("Delete Trainer")
        delete_btn.setStyleSheet("background-color: #e74c3c; color: white; padding:
8px; font-weight: bold;")
        delete_btn.clicked.connect(self.delete_trainer)
        buttons_layout.addWidget(delete_btn)
        clear_btn = QPushButton("Clear Form")
        clear_btn.setStyleSheet("background-color: #95a5a6; color: white; padding:
8px; font-weight: bold;")
        clear_btn.clicked.connect(self.clear_form)
        buttons_layout.addWidget(clear_btn)
        main_layout.addLayout(buttons_layout)
        self.table = QTableWidget()
        self.table.setColumnCount(6)
        self.table.setHorizontalHeaderLabels([
            "ID", "Name", "Specialization", "Phone", "Email", "Salary (₹)"
        ])
        self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
```

```
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        self.table.cellClicked.connect(self.table_clicked)
        main_layout.addWidget(self.table)
        self.setLayout(main_layout)
    def load_trainers(self):
        trainers = self.db.get_all_trainers()
        self.table.setRowCount(len(trainers))
        for row, trainer in enumerate(trainers):
            for col, value in enumerate(trainer):
                item = QTableWidgetItem(str(value) if value else "N/A")
                item.setTextAlignment(Qt.AlignCenter)
                self.table.setItem(row, col, item)
    def add_trainer(self):
        try:
            name = self.name_input.text().strip()
            specialization = self.specialization_input.text().strip()
            phone = self.phone_input.text().strip()
            email = self.email_input.text().strip()
            salary = float(self.salary_input.text())
            if not name:
                QMessageBox.warning(self, "Error", "Please enter trainer name!")
                return
            self.db.add_trainer(name, specialization, phone, email, salary)
            QMessageBox.information(self, "Success", "Trainer added successfully!")
            self.load_trainers()
            self.clear_form()
        except ValueError:
            QMessageBox.warning(self, "Error", "Please enter valid salary amount!")
        except Exception as e:
            QMessageBox.critical(self, "Error", f"Failed to add trainer: {str(e)}")
    def update_trainer(self):
        try:
            trainer_id = self.trainer_id_input.text()
            if not trainer_id:
                QMessageBox.warning(self, "Error", "Please select a trainer to update!")
                return
            name = self.name_input.text().strip()
            specialization = self.specialization_input.text().strip()
            phone = self.phone_input.text().strip()
            email = self.email_input.text().strip()
            salary = float(self.salary_input.text())
            self.db.update_trainer(trainer_id, name, specialization, phone, email, salary)
            QMessageBox.information(self, "Success", "Trainer updated successfully!")
```

```
            self.load_trainers()
            self.clear_form()
        except ValueError:
            QMessageBox.warning(self, "Error", "Please enter valid salary amount!")
        except Exception as e:
            QMessageBox.critical(self, "Error", f"Failed to update trainer: {str(e)}")
    def delete_trainer(self):
        trainer_id = self.trainer_id_input.text()
        if not trainer_id:
            QMessageBox.warning(self, "Error", "Please select a trainer to delete!")
            return
        reply = QMessageBox.question(self, "Confirm Delete",
                                     "Are you sure you want to delete this trainer?",
                                     QMessageBox.Yes | QMessageBox.No)
        if reply == QMessageBox.Yes:
            try:
                self.db.delete_trainer(trainer_id)
                QMessageBox.information(self, "Success", "Trainer deleted
successfully!")
                self.load_trainers()
                self.clear_form()
            except Exception as e:
                QMessageBox.critical(self, "Error", f"Failed to delete trainer: {str(e)}")
    def table_clicked(self, row):
        self.trainer_id_input.setText(self.table.item(row, 0).text())
        self.name_input.setText(self.table.item(row, 1).text())
        self.specialization_input.setText(self.table.item(row, 2).text())
        self.phone_input.setText(self.table.item(row, 3).text())
        self.email_input.setText(self.table.item(row, 4).text())
        self.salary_input.setText(self.table.item(row, 5).text())
    def clear_form(self):
        self.trainer_id_input.clear()
        self.name_input.clear()
        self.specialization_input.clear()
        self.phone_input.clear()
        self.email_input.clear()
        self.salary_input.clear()
```

metrics.py

```
from PyQt5.QtWidgets import (QWidget, QLabel, QLineEdit, QPushButton,
QTableWidget,
                             QTableWidgetItem, QVBoxLayout, QHBoxLayout,
QGridLayout,
```

```
                                QComboBox, QMessageBox, QHeaderView)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from database import Database
import matplotlib.pyplot as plt
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as
FigureCanvas
from matplotlib.figure import Figure
class MetricsWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.db = Database()
        self.init_ui()
        self.load_members()
    def init_ui(self):
        self.setWindowTitle("Body Metrics Tracking")
        self.setGeometry(100, 100, 1200, 700)
        main_layout = QVBoxLayout()
        header = QLabel("Body Metrics Tracking")
        header.setFont(QFont("Arial", 18, QFont.Bold))
        header.setStyleSheet("color: #2c3e50; padding: 10px;")
        header.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(header)
        form_layout = QGridLayout()
        self.member_combo = QComboBox()
        self.member_combo.currentIndexChanged.connect(self.load_metrics)
        self.weight_input = QLineEdit()
        self.weight_input.setPlaceholderText("Weight in kg")
        self.height_input = QLineEdit()
        self.height_input.setPlaceholderText("Height in cm")
        self.bmi_display = QLineEdit()
        self.bmi_display.setPlaceholderText("Auto-calculated")
        self.bmi_display.setReadOnly(True)
        form_layout.addWidget(QLabel("Select Member:"), 0, 0)
        form_layout.addWidget(self.member_combo, 0, 1)
        form_layout.addWidget(QLabel("Weight (kg):"), 0, 2)
        form_layout.addWidget(self.weight_input, 0, 3)
        form_layout.addWidget(QLabel("Height (cm):"), 1, 0)
        form_layout.addWidget(self.height_input, 1, 1)
        form_layout.addWidget(QLabel("BMI:"), 1, 2)
        form_layout.addWidget(self.bmi_display, 1, 3)
        main_layout.addLayout(form_layout)
        buttons_layout = QHBoxLayout()
```

```
        add_btn = QPushButton("Add Metric")
        add_btn.setStyleSheet("background-color: #27ae60; color: white; padding:
8px; font-weight: bold;")
        add_btn.clicked.connect(self.add_metric)
        buttons_layout.addWidget(add_btn)
        calculate_btn = QPushButton("Calculate BMI")
        calculate_btn.setStyleSheet("background-color: #3498db; color: white;
padding: 8px; font-weight: bold;")
        calculate_btn.clicked.connect(self.calculate_bmi)
        buttons_layout.addWidget(calculate_btn)
        show_chart_btn = QPushButton("Show Progress Chart")
        show_chart_btn.setStyleSheet("background-color: #9b59b6; color: white;
padding: 8px; font-weight: bold;")
        show_chart_btn.clicked.connect(self.show_chart)
        buttons_layout.addWidget(show_chart_btn)
        clear_btn = QPushButton("Clear Form")
        clear_btn.setStyleSheet("background-color: #95a5a6; color: white; padding:
8px; font-weight: bold;")
        clear_btn.clicked.connect(self.clear_form)
        buttons_layout.addWidget(clear_btn)
        main_layout.addLayout(buttons_layout)
        self.table = QTableWidget()
        self.table.setColumnCount(5)
        self.table.setHorizontalHeaderLabels([
            "Record ID", "Date", "Weight (kg)", "Height (cm)", "BMI"
        ])
        self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        main_layout.addWidget(self.table)

        info_label = QLabel(
            "BMI Categories: Underweight (<18.5) | Normal (18.5-24.9) | "
            "Overweight (25-29.9) | Obese (≥30)"
        )
        info_label.setStyleSheet("color: #7f8c8d; font-size: 10px; padding: 5px;")
        info_label.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(info_label)
        self.setLayout(main_layout)
    def load_members(self):
        self.member_combo.clear()
        self.member_combo.addItem("-- Select Member --", None)
        members = self.db.get_all_members()
        for member in members:
```

```
            self.member_combo.addItem(f"{member[1]} (ID: {member[0]})",
member[0])
    def load_metrics(self):
        member_id = self.member_combo.currentData()
        if member_id:
            metrics = self.db.get_member_metrics(member_id)
            self.table.setRowCount(len(metrics))
            for row, metric in enumerate(metrics):
                for col, value in enumerate(metric):
                    item = QTableWidgetItem(str(value))
                    item.setTextAlignment(Qt.AlignCenter)
                    self.table.setItem(row, col, item)
        else:
            self.table.setRowCount(0)
    def calculate_bmi(self):
        try:
            weight = float(self.weight_input.text())
            height = float(self.height_input.text())
            if height <= 0 or weight <= 0:
                QMessageBox.warning(self, "Error", "Weight and height must be
positive!")
                return
            height_m = height / 100
            bmi = weight / (height_m ** 2)
            self.bmi_display.setText(f"{bmi:.2f}")
            if bmi < 18.5:
                category = "Underweight"
            elif bmi < 25:
                category = "Normal"
            elif bmi < 30:
                category = "Overweight"
            else:
                category = "Obese"
            QMessageBox.information(self, "BMI Calculated",
                                    f"BMI: {bmi:.2f}\nCategory: {category}")
        except ValueError:
            QMessageBox.warning(self, "Error", "Please enter valid weight and
height!")
    def add_metric(self):
        try:
            member_id = self.member_combo.currentData()
            if not member_id:
                QMessageBox.warning(self, "Error", "Please select a member!")
```

```
            return
        weight = float(self.weight_input.text())
        height = float(self.height_input.text())
        if height <= 0 or weight <= 0:
            QMessageBox.warning(self, "Error", "Weight and height must be
positive!")
            return
        self.db.add_body_metric(member_id, weight, height)
        QMessageBox.information(self, "Success", "Body metric added
successfully!")
        self.load_metrics()
        self.clear_form()
    except ValueError:
        QMessageBox.warning(self, "Error", "Please enter valid weight and
height!")
    except Exception as e:
        QMessageBox.critical(self, "Error", f"Failed to add metric: {str(e)}")
  def show_chart(self):
    member_id = self.member_combo.currentData()
    if not member_id:
        QMessageBox.warning(self, "Error", "Please select a member!")
        return
    metrics = self.db.get_member_metrics(member_id)
    if not metrics:
        QMessageBox.information(self, "No Data", "No metrics found for this
member!")
        return
    metrics = list(reversed(metrics))
    dates = [m[1] for m in metrics]
    weights = [m[2] for m in metrics]
    bmis = [m[4] for m in metrics]
    chart_window = QWidget()
    chart_window.setWindowTitle(f"Progress Chart -
{self.member_combo.currentText()}")
    chart_window.setGeometry(200, 200, 900, 600)
    layout = QVBoxLayout()
    fig = Figure(figsize=(10, 6))
    ax1 = fig.add_subplot(2, 1, 1)
    ax1.plot(dates, weights, marker='o', linewidth=2, markersize=8,
color='#3498db')
    ax1.set_xlabel('Date', fontsize=12)
    ax1.set_ylabel('Weight (kg)', fontsize=12)
    ax1.set_title('Weight Progress', fontsize=14, fontweight='bold')
```

```
        ax1.grid(True, alpha=0.3)
        ax1.tick_params(axis='x', rotation=45)
        ax2 = fig.add_subplot(2, 1, 2)
        ax2.plot(dates, bmis, marker='s', linewidth=2, markersize=8, color='#e74c3c')
        ax2.set_xlabel('Date', fontsize=12)
        ax2.set_ylabel('BMI', fontsize=12)
        ax2.set_title('BMI Progress', fontsize=14, fontweight='bold')
        ax2.grid(True, alpha=0.3)
        ax2.tick_params(axis='x', rotation=45)
        ax2.axhspan(0, 18.5, alpha=0.2, color='blue', label='Underweight')
        ax2.axhspan(18.5, 25, alpha=0.2, color='green', label='Normal')
        ax2.axhspan(25, 30, alpha=0.2, color='orange', label='Overweight')
        ax2.axhspan(30, 50, alpha=0.2, color='red', label='Obese')
        ax2.legend(loc='upper right')
        fig.tight_layout()
        canvas = FigureCanvas(fig)
        layout.addWidget(canvas)
        chart_window.setLayout(layout)
        chart_window.show()
        self.chart_window = chart_window
    def clear_form(self):
        self.weight_input.clear()
        self.height_input.clear()
        self.bmi_display.clear()
```

plans.py

```
from PyQt5.QtWidgets import (QWidget, QLabel, QLineEdit, QPushButton,
QTableWidget,
                             QTableWidgetItem, QVBoxLayout, QHBoxLayout,
QGridLayout,
                             QComboBox, QMessageBox, QHeaderView, QSpinBox)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from database import Database
class WorkoutPlansWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.db = Database()
        self.init_ui()
        self.load_members()
        self.load_trainers()
        self.load_plans()
    def init_ui(self):
```

```
        self.setWindowTitle("Workout Plans Management")
        self.setGeometry(100, 100, 1000, 600)

        main_layout = QVBoxLayout()
        header = QLabel("Workout Plans Management")
        header.setFont(QFont("Arial", 18, QFont.Bold))
        header.setStyleSheet("color: #2c3e50; padding: 10px;")
        header.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(header)
        form_layout = QGridLayout()
        self.member_combo = QComboBox()
        self.trainer_combo = QComboBox()
        self.goal_input = QLineEdit()
        self.goal_input.setPlaceholderText("E.g., Weight Loss, Muscle Gain,
Endurance")
        self.level_combo = QComboBox()
        self.level_combo.addItems(["Beginner", "Intermediate", "Advanced", "Elite"])
        self.duration_spin = QSpinBox()
        self.duration_spin.setMinimum(1)
        self.duration_spin.setMaximum(52)
        self.duration_spin.setValue(12)
        self.duration_spin.setSuffix(" weeks")
        form_layout.addWidget(QLabel("Select Member:"), 0, 0)
        form_layout.addWidget(self.member_combo, 0, 1)
        form_layout.addWidget(QLabel("Assign Trainer:"), 0, 2)
        form_layout.addWidget(self.trainer_combo, 0, 3)
        form_layout.addWidget(QLabel("Goal:"), 1, 0)
        form_layout.addWidget(self.goal_input, 1, 1)
        form_layout.addWidget(QLabel("Level:"), 1, 2)
        form_layout.addWidget(self.level_combo, 1, 3)
        form_layout.addWidget(QLabel("Duration:"), 2, 0)
        form_layout.addWidget(self.duration_spin, 2, 1)
        main_layout.addLayout(form_layout)
        buttons_layout = QHBoxLayout()
        add_btn = QPushButton("Create Plan")
        add_btn.setStyleSheet("background-color: #27ae60; color: white; padding:
8px; font-weight: bold;")
        add_btn.clicked.connect(self.add_plan)
        buttons_layout.addWidget(add_btn)
        clear_btn = QPushButton("Clear Form")
        clear_btn.setStyleSheet("background-color: #95a5a6; color: white; padding:
8px; font-weight: bold;")
        clear_btn.clicked.connect(self.clear_form)
```

```
        buttons_layout.addWidget(clear_btn)
        refresh_btn = QPushButton("Refresh")
        refresh_btn.setStyleSheet("background-color: #3498db; color: white; padding:
8px; font-weight: bold;")
        refresh_btn.clicked.connect(self.load_plans)
        buttons_layout.addWidget(refresh_btn)
        main_layout.addLayout(buttons_layout)
        self.table = QTableWidget()
        self.table.setColumnCount(6)
        self.table.setHorizontalHeaderLabels([
            "Plan ID", "Member", "Trainer", "Goal", "Level", "Duration (weeks)"
        ])
        self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        main_layout.addWidget(self.table)
        info_label = QLabel(
            "💡 Tip: Workout plans help trainers customize training programs for
members based on their goals and fitness level."
        )
        info_label.setStyleSheet("color: #7f8c8d; font-size: 10px; padding: 5px;")
        info_label.setWordWrap(True)
        main_layout.addWidget(info_label)
        self.setLayout(main_layout)
    def load_members(self):
        self.member_combo.clear()
        self.member_combo.addItem("-- Select Member --", None)
        members = self.db.get_all_members()
        for member in members:
            self.member_combo.addItem(f"{member[1]} (ID: {member[0]})",
member[0])
    def load_trainers(self):
        self.trainer_combo.clear()
        self.trainer_combo.addItem("-- Select Trainer --", None)
        trainers = self.db.get_all_trainers()
        for trainer in trainers:
            self.trainer_combo.addItem(
                f"{trainer[1]} - {trainer[2]}",
                trainer[0]
            )
    def load_plans(self):
        plans = self.db.get_all_workout_plans()
        self.table.setRowCount(len(plans))
        for row, plan in enumerate(plans):
```

```
            for col, value in enumerate(plan):
                item = QTableWidgetItem(str(value) if value else "N/A")
                item.setTextAlignment(Qt.AlignCenter)
                self.table.setItem(row, col, item)
    def add_plan(self):
        try:
            member_id = self.member_combo.currentData()
            trainer_id = self.trainer_combo.currentData()
            goal = self.goal_input.text().strip()
            level = self.level_combo.currentText()
            duration = self.duration_spin.value()
            if not member_id:
                QMessageBox.warning(self, "Error", "Please select a member!")
                return
            if not trainer_id:
                QMessageBox.warning(self, "Error", "Please select a trainer!")
                return
            if not goal:
                QMessageBox.warning(self, "Error", "Please enter a goal!")
                return
            self.db.add_workout_plan(member_id, trainer_id, goal, level, duration)
            QMessageBox.information(self, "Success", "Workout plan created
successfully!")
            self.load_plans()
            self.clear_form()
        except Exception as e:
            QMessageBox.critical(self, "Error", f"Failed to create plan: {str(e)}")
    def clear_form(self):
        self.member_combo.setCurrentIndex(0)
        self.trainer_combo.setCurrentIndex(0)
        self.goal_input.clear()
        self.level_combo.setCurrentIndex(0)
        self.duration_spin.setValue(12)
```

payments.py

```
from PyQt5.QtWidgets import (QWidget, QLabel, QLineEdit, QPushButton,
QTableWidget,
                             QTableWidgetItem, QVBoxLayout, QHBoxLayout,
QGridLayout,
                             QComboBox, QMessageBox, QHeaderView)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from database import Database
```

```
from datetime import datetime, timedelta
class PaymentsWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.db = Database()
        self.init_ui()
        self.load_members()
        self.load_all_payments()
    def init_ui(self):
        self.setWindowTitle("Payments Management")
        self.setGeometry(100, 100, 1000, 600)
        main_layout = QVBoxLayout()
        header = QLabel("Payments Management")
        header.setFont(QFont("Arial", 18, QFont.Bold))
        header.setStyleSheet("color: #2c3e50; padding: 10px;")
        header.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(header)
        form_layout = QGridLayout()
        self.member_combo = QComboBox()
self.member_combo.currentIndexChanged.connect(self.load_member_payments)
        self.amount_input = QLineEdit()
        self.amount_input.setPlaceholderText("Enter payment amount")
        self.payment_mode_combo = QComboBox()
        self.payment_mode_combo.addItems([
            "Cash", "Credit Card", "Debit Card", "UPI", "Net Banking", "Cheque"
        ])
        self.status_label = QLabel("Status: Not Selected")
        self.status_label.setStyleSheet("padding: 5px; font-weight: bold;")
        form_layout.addWidget(QLabel("Select Member:"), 0, 0)
        form_layout.addWidget(self.member_combo, 0, 1)
        form_layout.addWidget(QLabel("Amount (₹):"), 0, 2)
        form_layout.addWidget(self.amount_input, 0, 3)
        form_layout.addWidget(QLabel("Payment Mode:"), 1, 0)
        form_layout.addWidget(self.payment_mode_combo, 1, 1)
        form_layout.addWidget(QLabel("Membership Status:"), 1, 2)
        form_layout.addWidget(self.status_label, 1, 3)
        main_layout.addLayout(form_layout)
        buttons_layout = QHBoxLayout()
        add_btn = QPushButton("Record Payment")
        add_btn.setStyleSheet("background-color: #27ae60; color: white; padding:
8px; font-weight: bold;")
        add_btn.clicked.connect(self.add_payment)
        buttons_layout.addWidget(add_btn)
```

```
        check_status_btn = QPushButton("Check Membership Status")
        check_status_btn.setStyleSheet("background-color: #3498db; color: white;
padding: 8px; font-weight: bold;")
        check_status_btn.clicked.connect(self.check_membership_status)
        buttons_layout.addWidget(check_status_btn)
        view_all_btn = QPushButton("View All Payments")
        view_all_btn.setStyleSheet("background-color: #9b59b6; color: white;
padding: 8px; font-weight: bold;")
        view_all_btn.clicked.connect(self.load_all_payments)
        buttons_layout.addWidget(view_all_btn)
        clear_btn = QPushButton("Clear Form")
        clear_btn.setStyleSheet("background-color: #95a5a6; color: white; padding:
8px; font-weight: bold;")
        clear_btn.clicked.connect(self.clear_form)
        buttons_layout.addWidget(clear_btn)
        main_layout.addLayout(buttons_layout)
        self.table = QTableWidget()
        self.table.setColumnCount(5)
        self.table.setHorizontalHeaderLabels([
            "Payment ID", "Member", "Date", "Amount (₹)", "Payment Mode"
        ])
        self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        main_layout.addWidget(self.table)
        self.setLayout(main_layout)
    def load_members(self):
        self.member_combo.clear()
        self.member_combo.addItem("-- Select Member --", None)
        members = self.db.get_all_members()
        for member in members:
            conn = self.db.get_connection()
            cursor = conn.cursor()
            cursor.execute('''
                SELECT m.member_id, m.join_date, mb.duration
                FROM members m
                LEFT JOIN memberships mb ON m.membership_id =
mb.membership_id
                WHERE m.member_id = ?
            ''', (member[0],))
            member_data = cursor.fetchone()
            conn.close()

            self.member_combo.addItem(
```

```
            f"{member[1]} (ID: {member[0]})",
            member_data
        )
    def load_member_payments(self):
        member_data = self.member_combo.currentData()
        if member_data:
            member_id = member_data[0]
            payments = self.db.get_member_payments(member_id)
            self.table.setRowCount(len(payments))
            for row, payment in enumerate(payments):
                member_name = self.member_combo.currentText().split(' (')[0]
                row_data = [
                    payment[0],
                    member_name,
                    payment[1],
                    payment[2],
                    payment[3]
                ]
                for col, value in enumerate(row_data):
                    item = QTableWidgetItem(str(value))
                    item.setTextAlignment(Qt.AlignCenter)
                    self.table.setItem(row, col, item)
        else:
            self.table.setRowCount(0)
    def load_all_payments(self):
        payments = self.db.get_all_payments()
        self.table.setRowCount(len(payments))
        for row, payment in enumerate(payments):
            for col, value in enumerate(payment):
                item = QTableWidgetItem(str(value))
                item.setTextAlignment(Qt.AlignCenter)
                self.table.setItem(row, col, item)
    def add_payment(self):
        try:
            member_data = self.member_combo.currentData()
            if not member_data:
                QMessageBox.warning(self, "Error", "Please select a member!")
                return
            member_id = member_data[0]
            amount = float(self.amount_input.text())
            payment_mode = self.payment_mode_combo.currentText()
            if amount <= 0:
                QMessageBox.warning(self, "Error", "Amount must be positive!")
```

```
                return
            self.db.add_payment(member_id, amount, payment_mode)
            QMessageBox.information(self, "Success", "Payment recorded
successfully!")
            self.load_member_payments()
            self.clear_form()
        except ValueError:
            QMessageBox.warning(self, "Error", "Please enter valid amount!")
        except Exception as e:
            QMessageBox.critical(self, "Error", f"Failed to record payment: {str(e)}")
    def check_membership_status(self):
        member_data = self.member_combo.currentData()
        if not member_data:
            QMessageBox.warning(self, "Error", "Please select a member!")
            return
        member_id, join_date, duration = member_data
        if not join_date or not duration:
            self.status_label.setText("Status: No Membership")
            self.status_label.setStyleSheet("color: #95a5a6; padding: 5px; font-weight:
bold;")
            return
        join_dt = datetime.strptime(join_date, '%Y-%m-%d')
        expiry_dt = join_dt + timedelta(days=duration)
        today = datetime.now()
        days_left = (expiry_dt - today).days
        if days_left > 0:
            self.status_label.setText(f"Status: Active ({days_left} days left)")
            self.status_label.setStyleSheet("color: #27ae60; padding: 5px; font-weight:
bold;")
        elif days_left >= -7:
            self.status_label.setText(f"Status: Expiring Soon ({abs(days_left)} days
ago)")
            self.status_label.setStyleSheet("color: #f39c12; padding: 5px; font-weight:
bold;")
        else:
            self.status_label.setText(f"Status: Expired ({abs(days_left)} days ago)")
            self.status_label.setStyleSheet("color: #e74c3c; padding: 5px; font-weight:
bold;")
    def clear_form(self):
        self.amount_input.clear()
        self.payment_mode_combo.setCurrentIndex(0)
        self.status_label.setText("Status: Not Selected")
        self.status_label.setStyleSheet("padding: 5px; font-weight: bold;")
```

memberships.py

```
from PyQt5.QtWidgets import (QWidget, QLabel, QLineEdit, QPushButton,
QTableWidget,
                             QTableWidgetItem, QVBoxLayout, QHBoxLayout,
QGridLayout,
                             QSpinBox, QMessageBox, QHeaderView)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from database import Database
class MembershipsWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.db = Database()
        self.init_ui()
        self.load_memberships()
    def init_ui(self):
        self.setWindowTitle("Membership Plans Management")
        self.setGeometry(100, 100, 800, 500)
        main_layout = QVBoxLayout()
        header = QLabel("Membership Plans Management")
        header.setFont(QFont("Arial", 18, QFont.Bold))
        header.setStyleSheet("color: #2c3e50; padding: 10px;")
        header.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(header)
        form_layout = QGridLayout()
        self.membership_id_input = QLineEdit()
        self.membership_id_input.setPlaceholderText("Auto")
        self.membership_id_input.setReadOnly(True)
        self.type_input = QLineEdit()
        self.type_input.setPlaceholderText("E.g., Monthly, Quarterly, Yearly")
        self.fee_input = QLineEdit()
        self.fee_input.setPlaceholderText("Enter membership fee")
        self.duration_spin = QSpinBox()
        self.duration_spin.setMinimum(1)
        self.duration_spin.setMaximum(365)
        self.duration_spin.setValue(30)
        self.duration_spin.setSuffix(" days")
        form_layout.addWidget(QLabel("Membership ID:"), 0, 0)
        form_layout.addWidget(self.membership_id_input, 0, 1)
        form_layout.addWidget(QLabel("Type:"), 0, 2)
        form_layout.addWidget(self.type_input, 0, 3)
        form_layout.addWidget(QLabel("Fee (₹):"), 1, 0)
```

```
        form_layout.addWidget(self.fee_input, 1, 1)
        form_layout.addWidget(QLabel("Duration:"), 1, 2)
        form_layout.addWidget(self.duration_spin, 1, 3)
        main_layout.addLayout(form_layout)
        buttons_layout = QHBoxLayout()
        add_btn = QPushButton("Add Plan")
        add_btn.setStyleSheet("background-color: #27ae60; color: white; padding:
8px; font-weight: bold;")
        add_btn.clicked.connect(self.add_membership)
        buttons_layout.addWidget(add_btn)
        update_btn = QPushButton("Update Plan")
        update_btn.setStyleSheet("background-color: #3498db; color: white; padding:
8px; font-weight: bold;")
        update_btn.clicked.connect(self.update_membership)
        buttons_layout.addWidget(update_btn)
        delete_btn = QPushButton("Delete Plan")
        delete_btn.setStyleSheet("background-color: #e74c3c; color: white; padding:
8px; font-weight: bold;")
        delete_btn.clicked.connect(self.delete_membership)
        buttons_layout.addWidget(delete_btn)
        clear_btn = QPushButton("Clear Form")
        clear_btn.setStyleSheet("background-color: #95a5a6; color: white; padding:
8px; font-weight: bold;")
        clear_btn.clicked.connect(self.clear_form)
        buttons_layout.addWidget(clear_btn)
        main_layout.addLayout(buttons_layout)
        self.table = QTableWidget()
        self.table.setColumnCount(4)
        self.table.setHorizontalHeaderLabels([
            "ID", "Type", "Fee (₹)", "Duration (days)"
        ])
        self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        self.table.cellClicked.connect(self.table_clicked)
        main_layout.addWidget(self.table)
        info_label = QLabel(
            "💡 Common Plans: Monthly (30 days), Quarterly (90 days), "
            "Half-Yearly (180 days), Yearly (365 days)"
        )
        info_label.setStyleSheet("color: #7f8c8d; font-size: 10px; padding: 5px;")
        info_label.setWordWrap(True)
        main_layout.addWidget(info_label)
        self.setLayout(main_layout)
```

```
    def load_memberships(self):
        memberships = self.db.get_all_memberships()
        self.table.setRowCount(len(memberships))
        for row, membership in enumerate(memberships):
            for col, value in enumerate(membership):
                item = QTableWidgetItem(str(value))
                item.setTextAlignment(Qt.AlignCenter)
                self.table.setItem(row, col, item)
    def add_membership(self):
        try:
            type_name = self.type_input.text().strip()
            fee = float(self.fee_input.text())
            duration = self.duration_spin.value()
            if not type_name:
                QMessageBox.warning(self, "Error", "Please enter membership type!")
                return
            if fee <= 0:
                QMessageBox.warning(self, "Error", "Fee must be positive!")
                return
            self.db.add_membership(type_name, fee, duration)
            QMessageBox.information(self, "Success", "Membership plan added
successfully!")
            self.load_memberships()
            self.clear_form()
        except ValueError:
            QMessageBox.warning(self, "Error", "Please enter valid fee amount!")
        except Exception as e:
            QMessageBox.critical(self, "Error", f"Failed to add membership: {str(e)}")
    def update_membership(self):
        try:
            membership_id = self.membership_id_input.text()
            if not membership_id:
                QMessageBox.warning(self, "Error", "Please select a membership to
update!")
                return
            type_name = self.type_input.text().strip()
            fee = float(self.fee_input.text())
            duration = self.duration_spin.value()

            if not type_name:
                QMessageBox.warning(self, "Error", "Please enter membership type!")
                return
            self.db.update_membership(membership_id, type_name, fee, duration)
```

```
            QMessageBox.information(self, "Success", "Membership plan updated
successfully!")
            self.load_memberships()
            self.clear_form()
        except ValueError:
            QMessageBox.warning(self, "Error", "Please enter valid fee amount!")
        except Exception as e:
            QMessageBox.critical(self, "Error", f"Failed to update membership:
{str(e)}")
    def delete_membership(self):
        membership_id = self.membership_id_input.text()
        if not membership_id:
            QMessageBox.warning(self, "Error", "Please select a membership to
delete!")
            return
        reply = QMessageBox.question(self, "Confirm Delete",
                                     "Are you sure you want to delete this membership plan?\n"
                                     "This may affect existing members!",
                                     QMessageBox.Yes | QMessageBox.No)
        if reply == QMessageBox.Yes:
            try:
                self.db.delete_membership(membership_id)
                QMessageBox.information(self, "Success", "Membership plan deleted
successfully!")
                self.load_memberships()
                self.clear_form()
            except Exception as e:
                QMessageBox.critical(self, "Error", f"Failed to delete membership:
{str(e)}")
    def table_clicked(self, row):
        self.membership_id_input.setText(self.table.item(row, 0).text())
        self.type_input.setText(self.table.item(row, 1).text())
        self.fee_input.setText(self.table.item(row, 2).text())
        self.duration_spin.setValue(int(self.table.item(row, 3).text()))
    def clear_form(self):
        self.membership_id_input.clear()
        self.type_input.clear()
        self.fee_input.clear()
        self.duration_spin.setValue(30)
```

equipment.py

```
from PyQt5.QtWidgets import (QWidget, QLabel, QLineEdit, QPushButton,
QTableWidget,
                             QTableWidgetItem, QVBoxLayout, QHBoxLayout,
QGridLayout,
                             QComboBox, QMessageBox, QHeaderView, QDateEdit)
from PyQt5.QtCore import Qt, QDate
from PyQt5.QtGui import QFont
from database import Database
class EquipmentWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.db = Database()
        self.init_ui()
        self.load_equipment()
    def init_ui(self):
        self.setWindowTitle("Equipment Management")
        self.setGeometry(100, 100, 1000, 600)
        main_layout = QVBoxLayout()
        header = QLabel("Equipment Management")
        header.setFont(QFont("Arial", 18, QFont.Bold))
        header.setStyleSheet("color: #2c3e50; padding: 10px;")
        header.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(header)
        form_layout = QGridLayout()
        self.equipment_id_input = QLineEdit()
        self.equipment_id_input.setPlaceholderText("Auto")
        self.equipment_id_input.setReadOnly(True)
        self.name_input = QLineEdit()
        self.name_input.setPlaceholderText("E.g., Treadmill, Dumbbell Set")
        self.type_combo = QComboBox()
        self.type_combo.addItems([
            "Cardio", "Strength", "Free Weights", "Machines",
            "Accessories", "Functional", "Other"
        ])
        self.purchase_date_input = QDateEdit()
        self.purchase_date_input.setDate(QDate.currentDate())
        self.purchase_date_input.setCalendarPopup(True)
        self.purchase_date_input.setDisplayFormat("yyyy-MM-dd")
        self.condition_combo = QComboBox()
        self.condition_combo.addItems([
            "Excellent", "Good", "Fair", "Needs Repair", "Out of Service"
        ])
```

```
        form_layout.addWidget(QLabel("Equipment ID:"), 0, 0)
        form_layout.addWidget(self.equipment_id_input, 0, 1)
        form_layout.addWidget(QLabel("Name:"), 0, 2)
        form_layout.addWidget(self.name_input, 0, 3)
        form_layout.addWidget(QLabel("Type:"), 1, 0)
        form_layout.addWidget(self.type_combo, 1, 1)
        form_layout.addWidget(QLabel("Purchase Date:"), 1, 2)
        form_layout.addWidget(self.purchase_date_input, 1, 3)
        form_layout.addWidget(QLabel("Condition:"), 2, 0)
        form_layout.addWidget(self.condition_combo, 2, 1)
        main_layout.addLayout(form_layout)
        buttons_layout = QHBoxLayout()
        add_btn = QPushButton("Add Equipment")
        add_btn.setStyleSheet("background-color: #27ae60; color: white; padding:
8px; font-weight: bold;")
        add_btn.clicked.connect(self.add_equipment)
        buttons_layout.addWidget(add_btn)
        update_btn = QPushButton("Update Equipment")
        update_btn.setStyleSheet("background-color: #3498db; color: white; padding:
8px; font-weight: bold;")
        update_btn.clicked.connect(self.update_equipment)
        buttons_layout.addWidget(update_btn)
        delete_btn = QPushButton("Delete Equipment")
        delete_btn.setStyleSheet("background-color: #e74c3c; color: white; padding:
8px; font-weight: bold;")
        delete_btn.clicked.connect(self.delete_equipment)
        buttons_layout.addWidget(delete_btn)
        clear_btn = QPushButton("Clear Form")
        clear_btn.setStyleSheet("background-color: #95a5a6; color: white; padding:
8px; font-weight: bold;")
        clear_btn.clicked.connect(self.clear_form)
        buttons_layout.addWidget(clear_btn)
        main_layout.addLayout(buttons_layout)
        self.table = QTableWidget()
        self.table.setColumnCount(5)
        self.table.setHorizontalHeaderLabels([
            "ID", "Name", "Type", "Purchase Date", "Condition"
        ])
        self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        self.table.cellClicked.connect(self.table_clicked)
        main_layout.addWidget(self.table)
        self.summary_label = QLabel()
```

```
        self.summary_label.setStyleSheet("color: #2c3e50; padding: 5px; font-weight:
bold;")
        self.summary_label.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(self.summary_label)
        self.setLayout(main_layout)
    def load_equipment(self):
        equipment_list = self.db.get_all_equipment()
        self.table.setRowCount(len(equipment_list))
        condition_counts = {}
        for row, equipment in enumerate(equipment_list):
            for col, value in enumerate(equipment):
                item = QTableWidgetItem(str(value) if value else "N/A")
                item.setTextAlignment(Qt.AlignCenter)
                if col == 4:
                    condition = str(value)
                    condition_counts[condition] = condition_counts.get(condition, 0) + 1
                    if condition == "Excellent":
                        item.setBackground(Qt.green)
                    elif condition == "Good":
                        item.setBackground(Qt.lightGray)
                    elif condition == "Fair":
                        item.setBackground(Qt.yellow)
                    elif condition == "Needs Repair":
                        item.setBackground(Qt.darkYellow)
                    elif condition == "Out of Service":
                        item.setBackground(Qt.red)
                self.table.setItem(row, col, item)
        total = len(equipment_list)
        self.summary_label.setText(
            f"Total Equipment: {total} | "
            f"Excellent: {condition_counts.get('Excellent', 0)} | "
            f"Good: {condition_counts.get('Good', 0)} | "
            f"Fair: {condition_counts.get('Fair', 0)} | "
            f"Needs Repair: {condition_counts.get('Needs Repair', 0)} | "
            f"Out of Service: {condition_counts.get('Out of Service', 0)}"
        )
    def add_equipment(self):
        try:
            name = self.name_input.text().strip()
            eq_type = self.type_combo.currentText()
            purchase_date = self.purchase_date_input.date().toString("yyyy-MM-dd")
            condition = self.condition_combo.currentText()
            if not name:
```

```
            QMessageBox.warning(self, "Error", "Please enter equipment name!")
            return
        self.db.add_equipment(name, eq_type, purchase_date, condition)
        QMessageBox.information(self, "Success", "Equipment added
successfully!")
        self.load_equipment()
        self.clear_form()
    except Exception as e:
        QMessageBox.critical(self, "Error", f"Failed to add equipment: {str(e)}")
def update_equipment(self):
    try:
        equipment_id = self.equipment_id_input.text()
        if not equipment_id:
            QMessageBox.warning(self, "Error", "Please select equipment to
update!")
            return
        name = self.name_input.text().strip()
        eq_type = self.type_combo.currentText()
        purchase_date = self.purchase_date_input.date().toString("yyyy-MM-dd")
        condition = self.condition_combo.currentText()
        if not name:
            QMessageBox.warning(self, "Error", "Please enter equipment name!")
            return
        self.db.update_equipment(equipment_id, name, eq_type, purchase_date,
condition)
        QMessageBox.information(self, "Success", "Equipment updated
successfully!")
        self.load_equipment()
        self.clear_form()
    except Exception as e:
        QMessageBox.critical(self, "Error", f"Failed to update equipment: {str(e)}")
def delete_equipment(self):
    equipment_id = self.equipment_id_input.text()
    if not equipment_id:
        QMessageBox.warning(self, "Error", "Please select equipment to delete!")
        return
    reply = QMessageBox.question(self, "Confirm Delete",
                                 "Are you sure you want to delete this equipment?",
                                 QMessageBox.Yes | QMessageBox.No)
    if reply == QMessageBox.Yes:
        try:
            self.db.delete_equipment(equipment_id)
            QMessageBox.information(self, "Success", "Equipment deleted
```

```
successfully!")
                self.load_equipment()
                self.clear_form()
            except Exception as e:
                QMessageBox.critical(self, "Error", f"Failed to delete equipment:
{str(e)}")
    def table_clicked(self, row):
        self.equipment_id_input.setText(self.table.item(row, 0).text())
        self.name_input.setText(self.table.item(row, 1).text())
        eq_type = self.table.item(row, 2).text()
        index = self.type_combo.findText(eq_type)
        if index >= 0:
            self.type_combo.setCurrentIndex(index)
        purchase_date_str = self.table.item(row, 3).text()
        if purchase_date_str != "N/A":
            date = QDate.fromString(purchase_date_str, "yyyy-MM-dd")
            self.purchase_date_input.setDate(date)
        condition = self.table.item(row, 4).text()
        index = self.condition_combo.findText(condition)
        if index >= 0:
            self.condition_combo.setCurrentIndex(index)
    def clear_form(self):
        self.equipment_id_input.clear()
        self.name_input.clear()
        self.type_combo.setCurrentIndex(0)
        self.purchase_date_input.setDate(QDate.currentDate())
        self.condition_combo.setCurrentIndex(0)
```

analytics.py

```
from PyQt5.QtWidgets import (QWidget, QLabel, QVBoxLayout, QHBoxLayout,
                             QGridLayout, QFrame, QPushButton, QScrollArea, QSizePolicy)
from PyQt5.QtCore import Qt, QTimer
from PyQt5.QtGui import QFont, QColor
from database import Database
import matplotlib.pyplot as plt
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as
FigureCanvas
from matplotlib.figure import Figure
import numpy as np
from datetime import datetime, timedelta
class AnalyticsWindow(QWidget):
    def __init__(self):
        super().__init__()
```

```
    self.db = Database()
    self.init_ui()
    self.timer = QTimer()
    self.timer.timeout.connect(self.refresh_all_data)
    self.timer.start(5000)
    self.refresh_all_data()
def init_ui(self):
    self.setWindowTitle("Real-Time Analytics Dashboard")
    self.setGeometry(50, 50, 1600, 900)
    self.setObjectName("AnalyticsMain")
    self.setStyleSheet("""
        QWidget#AnalyticsMain {
            background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
                stop:0 #0f2027, stop:0.5 #203a43, stop:1 #2c5364);
            color: #ecf0f1;
        }
    """)
    main_layout = QVBoxLayout(self)
    main_layout.setContentsMargins(30, 30, 30, 30)
    main_layout.setSpacing(25)
    header = self.create_header()
    main_layout.addWidget(header)
    kpi_layout = self.create_kpi_cards()
    main_layout.addLayout(kpi_layout)
    scroll = QScrollArea()
    scroll.setWidgetResizable(True)
    scroll.setFrameShape(QFrame.NoFrame)
    scroll.setStyleSheet("""
        QScrollArea {
            background: transparent;
            border: none;
        }
        QScrollBar:vertical {
            border: none;
            background: rgba(255, 255, 255, 0.05);
            width: 10px;
            border-radius: 5px;
        }
        QScrollBar::handle:vertical {
            background: rgba(255, 255, 255, 0.2);
            border-radius: 5px;
        }
        QScrollBar::handle:vertical:hover {
```

```
            background: rgba(0, 210, 255, 0.5);
        }
    """)
    scroll_widget = QWidget()
    scroll_widget.setStyleSheet("background: transparent;")
    scroll_layout = QVBoxLayout(scroll_widget)
    scroll_layout.setContentsMargins(0, 10, 0, 0)
    charts_grid = QGridLayout()
    charts_grid.setSpacing(25)
    for i in range(2):
        charts_grid.setColumnStretch(i, 1)
    self.revenue_chart = self.create_chart_widget("MONTHLY REVENUE
TREND", 0)
    self.membership_chart = self.create_chart_widget("MEMBERSHIP
DISTRIBUTION", 1)
    charts_grid.addWidget(self.revenue_chart, 0, 0)
    charts_grid.addWidget(self.membership_chart, 0, 1)

    self.growth_chart = self.create_chart_widget("MEMBER GROWTH OVER
TIME", 2)
    self.gender_chart = self.create_chart_widget("GENDER DISTRIBUTION", 3)
    charts_grid.addWidget(self.growth_chart, 1, 0)
    charts_grid.addWidget(self.gender_chart, 1, 1)
    self.trainer_chart = self.create_chart_widget("TRAINER WORKLOAD", 4)
    self.equipment_chart = self.create_chart_widget("EQUIPMENT CONDITION
STATUS", 5)
    charts_grid.addWidget(self.trainer_chart, 2, 0)
    charts_grid.addWidget(self.equipment_chart, 2, 1)
    self.payment_chart = self.create_chart_widget("PAYMENT METHODS
USAGE", 6)
    self.bmi_chart = self.create_chart_widget("MEMBER BMI DISTRIBUTION",
7)
    charts_grid.addWidget(self.payment_chart, 3, 0)
    charts_grid.addWidget(self.bmi_chart, 3, 1)
    scroll_layout.addLayout(charts_grid)
    scroll.setWidget(scroll_widget)
    main_layout.addWidget(scroll)
    footer = self.create_footer()
    main_layout.addWidget(footer)
  def create_header(self):
    header_frame = QFrame()
    header_frame.setStyleSheet("""
        QFrame {
```

```
            background: rgba(20, 25, 30, 0.7);
            border-radius: 16px;
            border: 1px solid rgba(255, 255, 255, 0.08);
        }
    """)
    header_frame.setFixedHeight(100)
    header_layout = QHBoxLayout(header_frame)
    header_layout.setContentsMargins(30, 0, 30, 0)
    title = QLabel("REAL-TIME ANALYTICS")
    title.setFont(QFont("Segoe UI Black", 24))
    title.setStyleSheet("color: white; background: transparent; border: none; letter-
spacing: 2px;")
    header_layout.addWidget(title)
    header_layout.addStretch()
    live_label = QLabel("● LIVE")
    live_label.setFont(QFont("Segoe UI", 11, QFont.Bold))
    live_label.setStyleSheet("""
        color: #ff4757;
        background: transparent;
        border: none;
    """)
    header_layout.addWidget(live_label)
    header_layout.addSpacing(15)
    refresh_btn = QPushButton("Refresh Data")
    refresh_btn.setFont(QFont("Segoe UI", 11, QFont.Bold))
    refresh_btn.setCursor(Qt.PointingHandCursor)
    refresh_btn.setStyleSheet("""
        QPushButton {
            background: qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 #0052D4,
stop:0.5 #4364F7, stop:1 #6FB1FC);
            color: white;
            border: none;
            padding: 10px 24px;
            border-radius: 16px;
        }
        QPushButton:hover {
            background: qlineargradient(x1:0, y1:0, x2:1, y2:0, stop:0 #6FB1FC,
stop:0.5 #4364F7, stop:1 #0052D4);
        }
    """)
    refresh_btn.clicked.connect(self.refresh_all_data)
    header_layout.addWidget(refresh_btn)
    return header_frame
```

```
    def create_kpi_cards(self):
        kpi_layout = QHBoxLayout()
        kpi_layout.setSpacing(20)
        self.kpi_cards = []
        kpi_data = [
            ("TOTAL MEMBERS", "0", "#00d2ff"),
            ("MONTHLY REVENUE", "₹0", "#38ef7d"),
            ("ACTIVE TRAINERS", "0", "#f64f59"),
            ("EQUIPMENT", "0", "#fbd786"),
            ("GROWTH RATE", "0%", "#c471ed")
        ]
        for title, value, color in kpi_data:
            card = self.create_kpi_card(title, value, color)
            self.kpi_cards.append(card)
            kpi_layout.addWidget(card['frame'])
        return kpi_layout
    def create_kpi_card(self, title, value, color):
        card_frame = QFrame()
        card_frame.setStyleSheet(f"""
            QFrame {{
                background: rgba(0, 0, 0, 0.4);
                border-radius: 16px;
                border: 1px solid rgba(255, 255, 255, 0.05);
                border-bottom: 3px solid {color};
            }}
            QFrame:hover {{
                background: rgba(20, 30, 40, 0.8);
            }}
        """)
        card_frame.setMinimumHeight(130)
        layout = QVBoxLayout(card_frame)
        layout.setContentsMargins(20, 20, 20, 20)
        layout.setAlignment(Qt.AlignCenter)
        value_label = QLabel(value)
        value_label.setFont(QFont("Segoe UI", 32, QFont.Bold))
        value_label.setStyleSheet("background: transparent; border: none; color:
white;")
        value_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(value_label)
        title_label = QLabel(title)
        title_label.setFont(QFont("Segoe UI", 10, QFont.Bold))
        title_label.setStyleSheet("background: transparent; border: none; color:
#a0a5a8; letter-spacing: 1px;")
```

```
        title_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(title_label)
        return {'frame': card_frame, 'value_label': value_label, 'title': title}
    def create_chart_widget(self, title, chart_id):
        frame = QFrame()
        frame.setStyleSheet("""
            QFrame {
                background: rgba(0, 0, 0, 0.4);
                border-radius: 16px;
                border: 1px solid rgba(255, 255, 255, 0.05);
            }
        """)
        frame.setMinimumHeight(400)
        layout = QVBoxLayout(frame)
        layout.setContentsMargins(20, 20, 20, 20)
        title_label = QLabel(title)
        title_label.setFont(QFont("Segoe UI", 11, QFont.Bold))
        title_label.setStyleSheet("color: #00d2ff; background: transparent; border:
none; letter-spacing: 1px;")
        layout.addWidget(title_label)
        layout.addSpacing(10)
        figure = Figure(figsize=(8, 4), facecolor='none')
        canvas = FigureCanvas(figure)
        canvas.setStyleSheet("background: transparent;")
        canvas.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
        layout.addWidget(canvas)
        frame.figure = figure
        frame.canvas = canvas
        frame.chart_id = chart_id
        return frame
    def create_footer(self):
        footer = QLabel("Last updated: Loading...")
        footer.setFont(QFont("Segoe UI", 9))
        footer.setStyleSheet("color: #6c757d; background: transparent;")
        footer.setAlignment(Qt.AlignCenter)
        self.footer_label = footer
        return footer
    def refresh_all_data(self):
        self.update_kpi_cards()
        self.update_revenue_chart()
        self.update_membership_chart()
        self.update_growth_chart()
        self.update_gender_chart()
```

```
        self.update_trainer_chart()
        self.update_equipment_chart()
        self.update_payment_chart()
        self.update_bmi_chart()
        self.footer_label.setText(f"Last updated: {datetime.now().strftime('%Y-%m-
%d %H:%M:%S')} • Auto-refreshing every 5 seconds")
    def update_kpi_cards(self):
        try:
            members = self.db.get_all_members()
            trainers = self.db.get_all_trainers()
            equipment = self.db.get_all_equipment()
            conn = self.db.get_connection()
            cursor = conn.cursor()
            cursor.execute("SELECT SUM(amount) FROM payments WHERE date >=
date('now', '-30 days')")
            monthly_revenue = cursor.fetchone()[0] or 0
            cursor.execute("SELECT COUNT(*) FROM members WHERE join_date
>= date('now', '-30 days')")
            recent_members = cursor.fetchone()[0] or 0
            cursor.execute("SELECT COUNT(*) FROM members WHERE join_date
>= date('now', '-60 days') AND join_date < date('now', '-30 days')")
            previous_members = cursor.fetchone()[0] or 1
            conn.close()
            growth_rate = ((recent_members - previous_members) / previous_members
* 100) if previous_members > 0 else 0
            self.kpi_cards[0]['value_label'].setText(str(len(members) if members else 0))
            self.kpi_cards[1]['value_label'].setText(f"₹{monthly_revenue:,.0f}")
            self.kpi_cards[2]['value_label'].setText(str(len(trainers) if trainers else 0))
            self.kpi_cards[3]['value_label'].setText(str(len(equipment) if equipment else
0))
            self.kpi_cards[4]['value_label'].setText(f"{growth_rate:+.1f}%")
        except Exception:
            pass
    def update_revenue_chart(self):
        try:
            conn = self.db.get_connection()
            cursor = conn.cursor()
            months, revenues = [], []
            for i in range(11, -1, -1):
                month_start = (datetime.now() - timedelta(days=30*i)).strftime('%Y-
%m')
                cursor.execute("SELECT SUM(amount) FROM payments WHERE
strftime('%Y-%m', date) = ?", (month_start,))
```

```
            revenue = cursor.fetchone()[0] or 0
            months.append(month_start)
            revenues.append(revenue)
        conn.close()
        fig = self.revenue_chart.figure
        fig.clear()
        ax = fig.add_subplot(111)
        ax.set_facecolor('none')
        fig.patch.set_facecolor('none')
        ax.fill_between(range(len(months)), revenues, alpha=0.3, color='#00d2ff')
        ax.plot(months, revenues, marker='o', linewidth=2, markersize=6,
color='#00d2ff', markerfacecolor='#ffffff')
        ax.tick_params(colors='#a0a5a8', labelsize=8)
        ax.grid(True, alpha=0.1, color='white')
        ax.spines['bottom'].set_color('#444')
        ax.spines['left'].set_color('#444')
        ax.spines['top'].set_visible(False)
        ax.spines['right'].set_visible(False)
        plt.setp(ax.xaxis.get_majorticklabels(), rotation=45, ha='right')
        fig.tight_layout()
        self.revenue_chart.canvas.draw()
    except Exception:
        pass
def update_membership_chart(self):
    try:
        conn = self.db.get_connection()
        cursor = conn.cursor()
        cursor.execute("""
            SELECT m.type, COUNT(mem.member_id)
            FROM memberships m LEFT JOIN members mem ON
m.membership_id = mem.membership_id
            GROUP BY m.type
        """)
        data = cursor.fetchall()
        conn.close()
        if not data or all(count == 0 for _, count in data): data = [("No Data", 1)]
        labels = [row[0] for row in data]
        sizes = [row[1] for row in data]
        fig = self.membership_chart.figure
        fig.clear()
        ax = fig.add_subplot(111)
        ax.set_facecolor('none')
        fig.patch.set_facecolor('none')
```

```
            colors = ['#00d2ff', '#38ef7d', '#f64f59', '#c471ed', '#fbd786', '#11998e']
            wedges, texts, autotexts = ax.pie(sizes, labels=labels, autopct='%1.1f%%',
colors=colors, startangle=90, textprops={'color': '#a0a5a8', 'fontsize': 9})
            for autotext in autotexts:
                autotext.set_color('white')
                autotext.set_fontweight('bold')
            fig.tight_layout()
            self.membership_chart.canvas.draw()
        except Exception:
            pass
    def update_growth_chart(self):
        try:
            conn = self.db.get_connection()
            cursor = conn.cursor()
            months, counts = [], []
            for i in range(11, -1, -1):
                month_start = (datetime.now() - timedelta(days=30*i)).strftime('%Y-
%m')
                cursor.execute("SELECT COUNT(*) FROM members WHERE
strftime('%Y-%m', join_date) <= ?", (month_start,))
                count = cursor.fetchone()[0] or 0
                months.append(month_start)
                counts.append(count)
            conn.close()
            fig = self.growth_chart.figure
            fig.clear()
            ax = fig.add_subplot(111)
            ax.set_facecolor('none')
            fig.patch.set_facecolor('none')
            ax.bar(months, counts, color='#c471ed', alpha=0.7)
            z = np.polyfit(range(len(counts)), counts, 1)
            p = np.poly1d(z)
            ax.plot(months, p(range(len(counts))), color='#ffffff', linestyle='--',
linewidth=2, alpha=0.8)
            ax.tick_params(colors='#a0a5a8', labelsize=8)
            ax.grid(True, alpha=0.1, color='white')
            ax.spines['bottom'].set_color('#444')
            ax.spines['left'].set_color('#444')
            ax.spines['top'].set_visible(False)
            ax.spines['right'].set_visible(False)
            plt.setp(ax.xaxis.get_majorticklabels(), rotation=45, ha='right')
            fig.tight_layout()
```

```
            self.growth_chart.canvas.draw()
        except Exception:
            pass
    def update_gender_chart(self):
        try:
            conn = self.db.get_connection()
            cursor = conn.cursor()
            cursor.execute("SELECT gender, COUNT(*) FROM members GROUP BY
gender")
            data = cursor.fetchall()
            conn.close()
            if not data: data = [("No Data", 1)]
            labels = [row[0] for row in data]
            sizes = [row[1] for row in data]
            fig = self.gender_chart.figure
            fig.clear()
            ax = fig.add_subplot(111)
            ax.set_facecolor('none')
            fig.patch.set_facecolor('none')
            colors = ['#00d2ff', '#f64f59', '#fbd786']
            wedges, texts, autotexts = ax.pie(sizes, labels=labels, autopct='%1.1f%%',
colors=colors, startangle=90, textprops={'color': '#a0a5a8', 'fontsize': 9},
pctdistance=0.85)
            centre_circle = plt.Circle((0, 0), 0.70, fc='#14191e')
            ax.add_artist(centre_circle)
            for autotext in autotexts:
                autotext.set_color('white')
                autotext.set_fontweight('bold')
            fig.tight_layout()
            self.gender_chart.canvas.draw()
        except Exception:
            pass
    def update_trainer_chart(self):
        try:
            conn = self.db.get_connection()
            cursor = conn.cursor()
            cursor.execute("""
                SELECT t.name, COUNT(m.member_id)
                FROM trainers t LEFT JOIN members m ON t.trainer_id = m.trainer_id
                GROUP BY t.trainer_id ORDER BY COUNT(m.member_id) DESC
LIMIT 10
            """)
            data = cursor.fetchall()
```

```
            conn.close()
            if not data: data = [("No Trainers", 0)]
            names = [row[0][:15] for row in data]
            counts = [row[1] for row in data]
            fig = self.trainer_chart.figure
            fig.clear()
            ax = fig.add_subplot(111)
            ax.set_facecolor('none')
            fig.patch.set_facecolor('none')
            ax.barh(names, counts, color='#38ef7d', alpha=0.7)
            ax.tick_params(colors='#a0a5a8', labelsize=8)
            ax.grid(True, alpha=0.1, color='white', axis='x')
            ax.spines['bottom'].set_color('#444')
            ax.spines['left'].set_color('#444')
            ax.spines['top'].set_visible(False)
            ax.spines['right'].set_visible(False)
            fig.tight_layout()
            self.trainer_chart.canvas.draw()
        except Exception:
            pass
    def update_equipment_chart(self):
        try:
            conn = self.db.get_connection()
            cursor = conn.cursor()
            cursor.execute("SELECT condition, COUNT(*) FROM equipment GROUP
BY condition")
            data = cursor.fetchall()
            conn.close()
            if not data: data = [("No Data", 1)]
            labels = [row[0] for row in data]
            sizes = [row[1] for row in data]
            fig = self.equipment_chart.figure
            fig.clear()
            ax = fig.add_subplot(111)
            ax.set_facecolor('none')
            fig.patch.set_facecolor('none')
            color_map = {'Excellent': '#38ef7d', 'Good': '#00d2ff', 'Fair': '#fbd786',
'Needs Repair': '#f64f59', 'Out of Service': '#eb3349'}
            colors = [color_map.get(label, '#95a5a6') for label in labels]
            ax.bar(labels, sizes, color=colors, alpha=0.8)
            ax.tick_params(colors='#a0a5a8', labelsize=8)
            ax.grid(True, alpha=0.1, color='white', axis='y')
            ax.spines['bottom'].set_color('#444')
```

```
            ax.spines['left'].set_color('#444')
            ax.spines['top'].set_visible(False)
            ax.spines['right'].set_visible(False)
            plt.setp(ax.xaxis.get_majorticklabels(), rotation=45, ha='right')
            fig.tight_layout()
            self.equipment_chart.canvas.draw()
        except Exception:
            pass
    def update_payment_chart(self):
        try:
            conn = self.db.get_connection()
            cursor = conn.cursor()
            cursor.execute("SELECT payment_mode, COUNT(*) FROM payments
GROUP BY payment_mode")
            data = cursor.fetchall()
            conn.close()
            if not data: data = [("No Data", 1)]
            labels = [row[0] for row in data]
            sizes = [row[1] for row in data]
            fig = self.payment_chart.figure
            fig.clear()
            ax = fig.add_subplot(111)
            ax.set_facecolor('none')
            fig.patch.set_facecolor('none')
            colors = ['#11998e', '#fbd786', '#00d2ff', '#c471ed']
            wedges, texts, autotexts = ax.pie(sizes, labels=labels, autopct='%1.1f%%',
colors=colors, startangle=90, textprops={'color': '#a0a5a8', 'fontsize': 9})
            for autotext in autotexts:
                autotext.set_color('white')
                autotext.set_fontweight('bold')
            fig.tight_layout()
            self.payment_chart.canvas.draw()
        except Exception:
            pass
    def update_bmi_chart(self):
        try:
            conn = self.db.get_connection()
            cursor = conn.cursor()
            cursor.execute("""
                SELECT bmi FROM body_metrics
                WHERE bmi IS NOT NULL AND record_id IN (SELECT
MAX(record_id) FROM body_metrics GROUP BY member_id)
            """)
```

```
            data = cursor.fetchall()
            conn.close()
            bmis = [row[0] for row in data] if data else [22.5]
            fig = self.bmi_chart.figure
            fig.clear()
            ax = fig.add_subplot(111)
            ax.set_facecolor('none')
            fig.patch.set_facecolor('none')
            n, bins, patches = ax.hist(bmis, bins=20, color='#00d2ff', alpha=0.7)
            for i, patch in enumerate(patches):
                if bins[i] < 18.5: patch.set_facecolor('#00d2ff')
                elif bins[i] < 25: patch.set_facecolor('#38ef7d')
                elif bins[i] < 30: patch.set_facecolor('#fbd786')
                else: patch.set_facecolor('#f64f59')
            ax.tick_params(colors='#a0a5a8', labelsize=8)
            ax.grid(True, alpha=0.1, color='white', axis='y')
            ax.spines['bottom'].set_color('#444')
            ax.spines['left'].set_color('#444')
            ax.spines['top'].set_visible(False)
            ax.spines['right'].set_visible(False)
            ax.axvline(18.5, color='white', linestyle='--', alpha=0.3, linewidth=1)
            ax.axvline(25, color='white', linestyle='--', alpha=0.3, linewidth=1)
            ax.axvline(30, color='white', linestyle='--', alpha=0.3, linewidth=1)
            fig.tight_layout()
            self.bmi_chart.canvas.draw()
        except Exception:
            pass
    def closeEvent(self, event):
        self.timer.stop()
        event.accept()
```

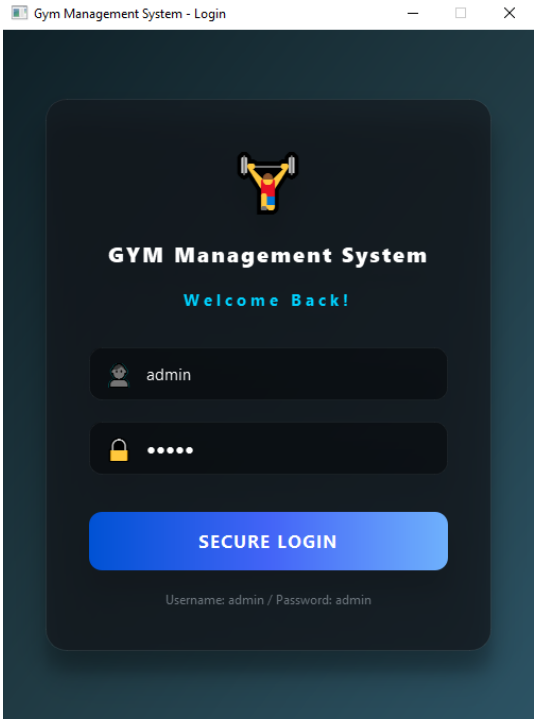


Fig. 5.1: Login.

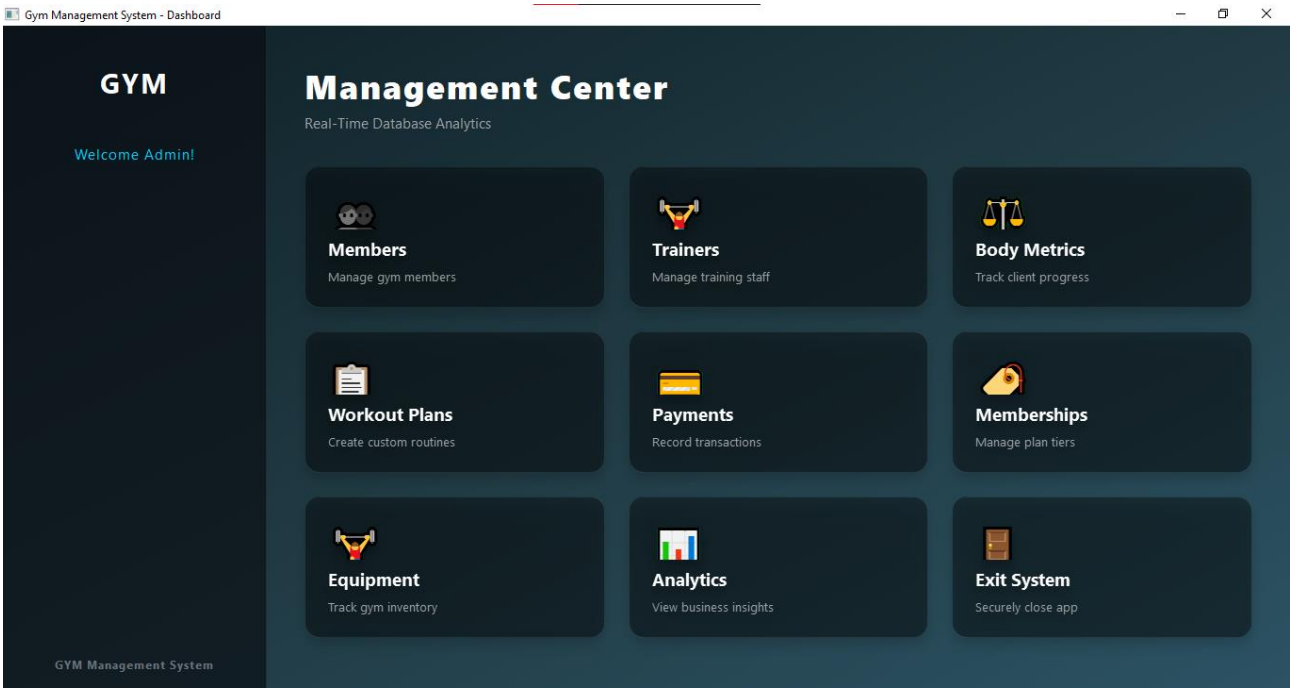


Fig. 5.2: Dashboard.

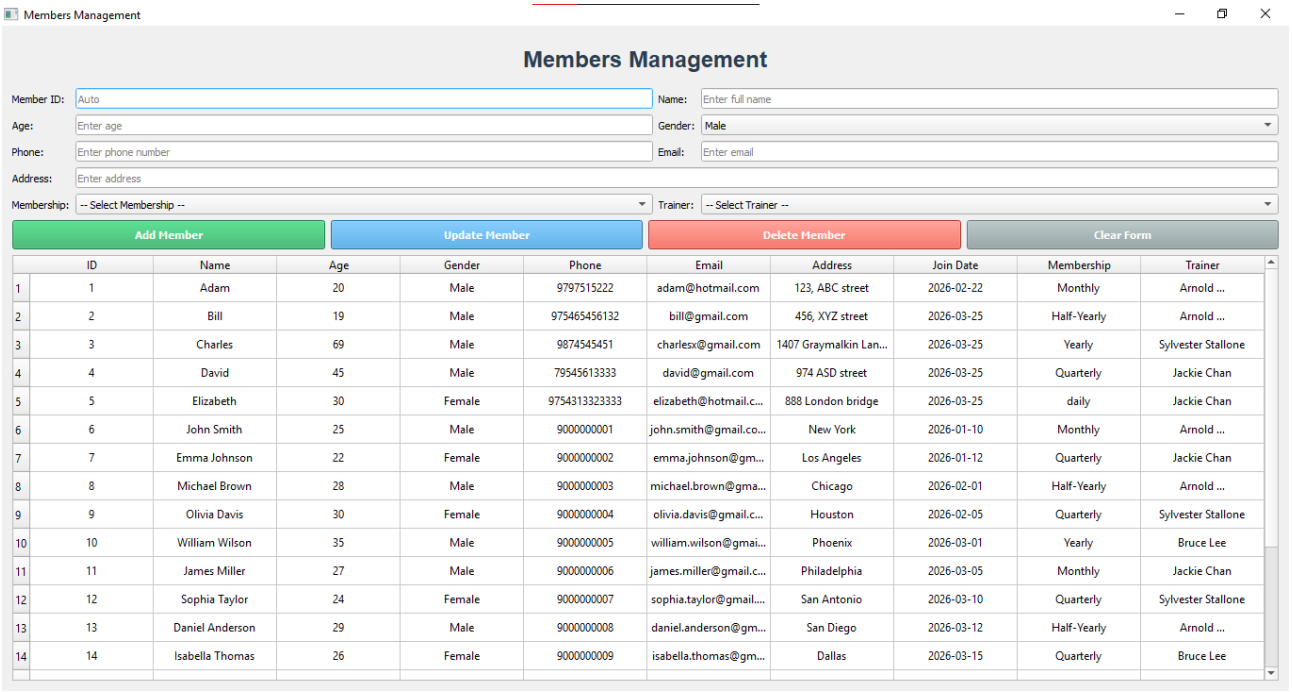


Fig. 5.3: Member Management.

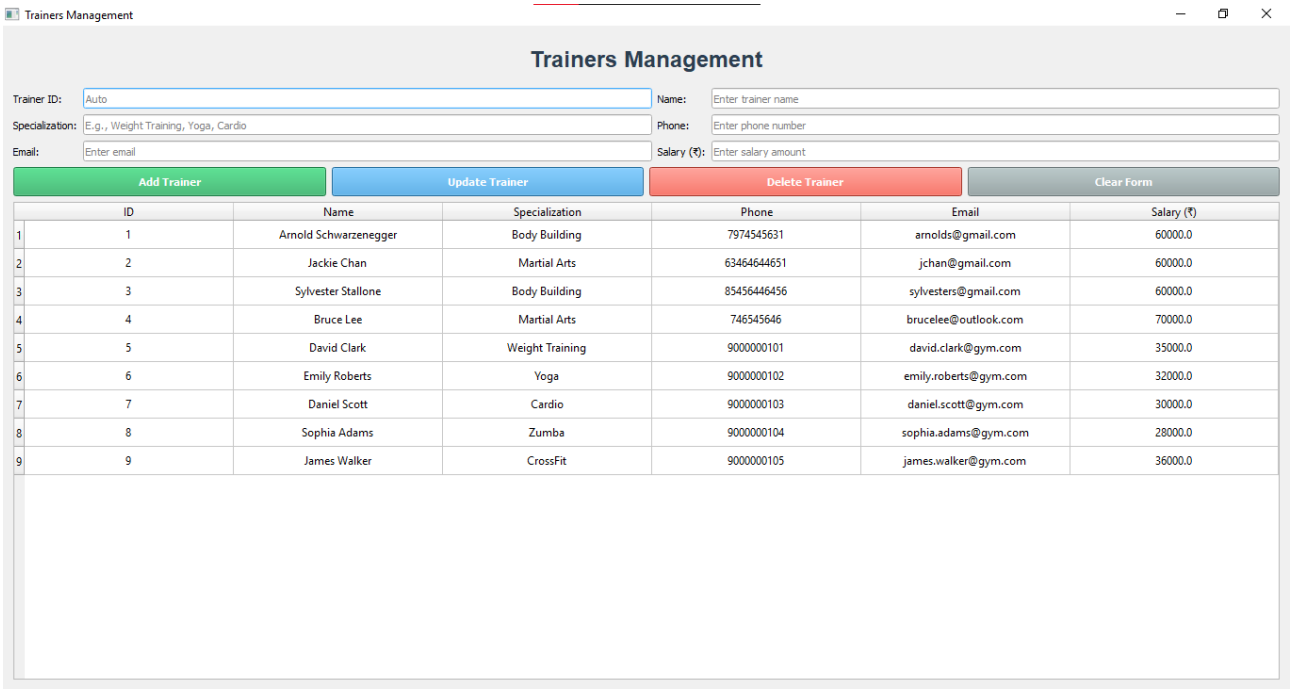


Fig. 5.4: Trainer Management.

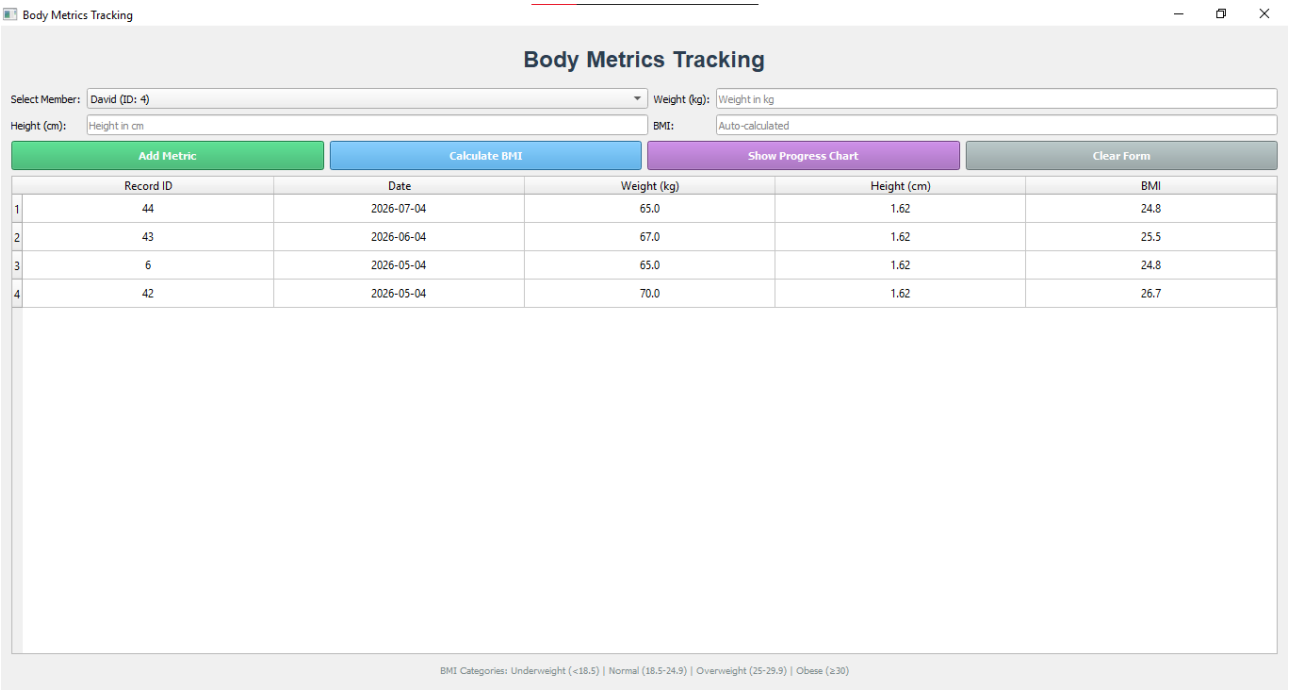


Fig. 5.5: Body Metrics Tracking.

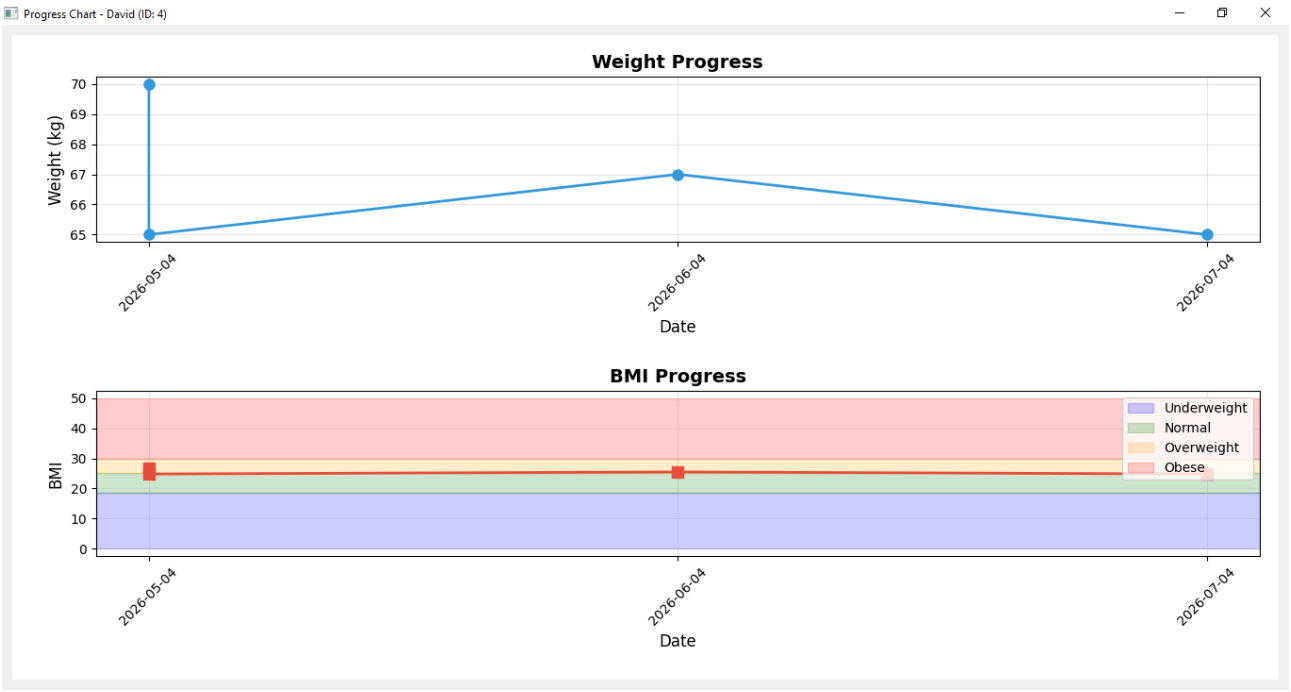


Fig. 5.6: Progress Chart.

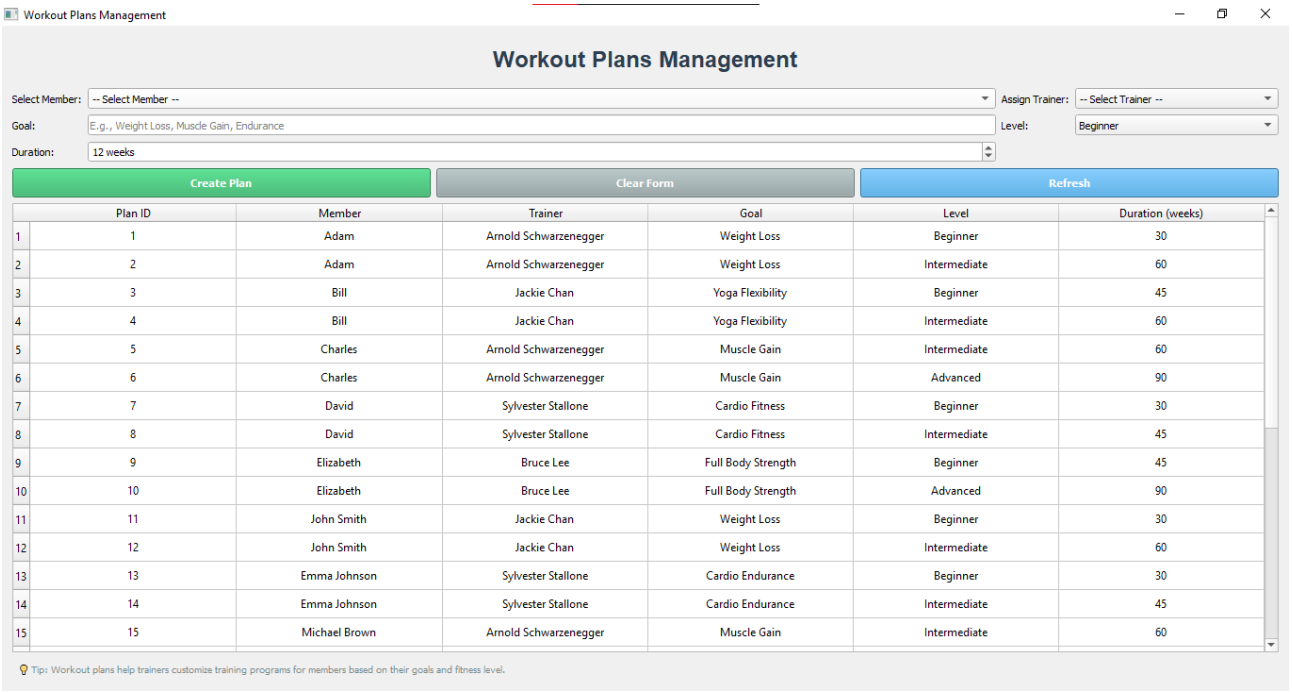


Fig. 5.7: Workout Plans Management.

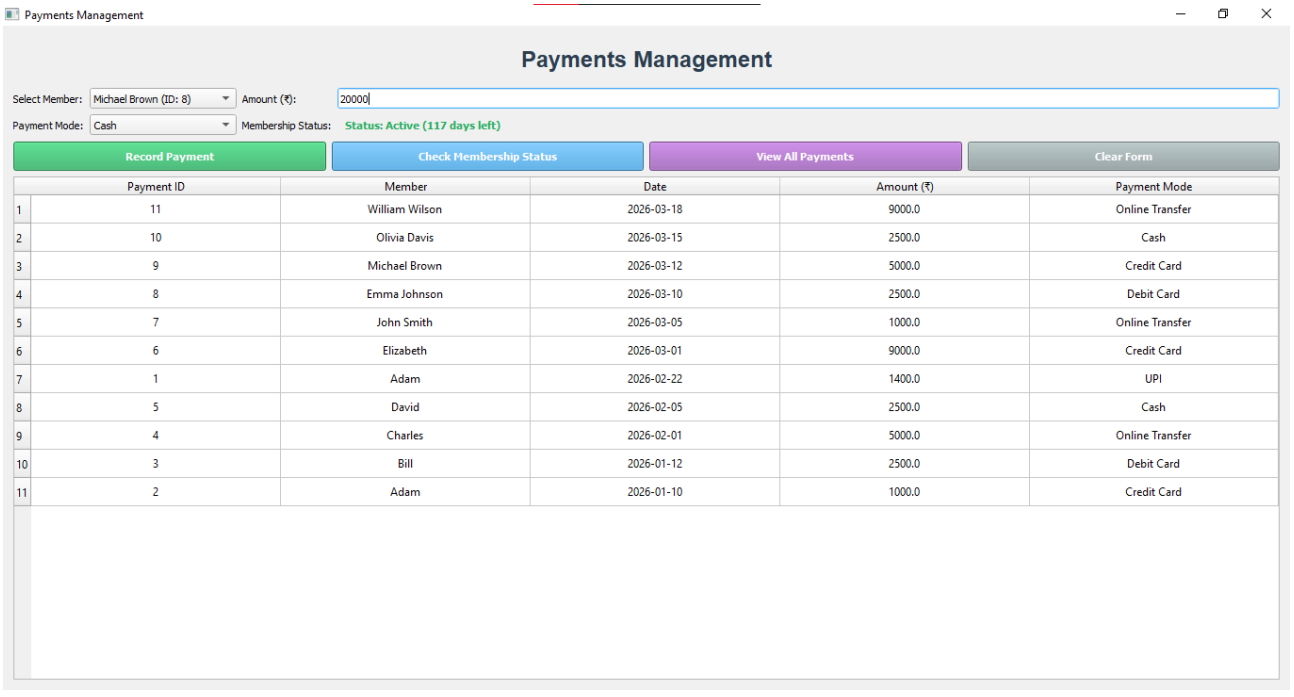


Fig. 5.8: Payments Management.

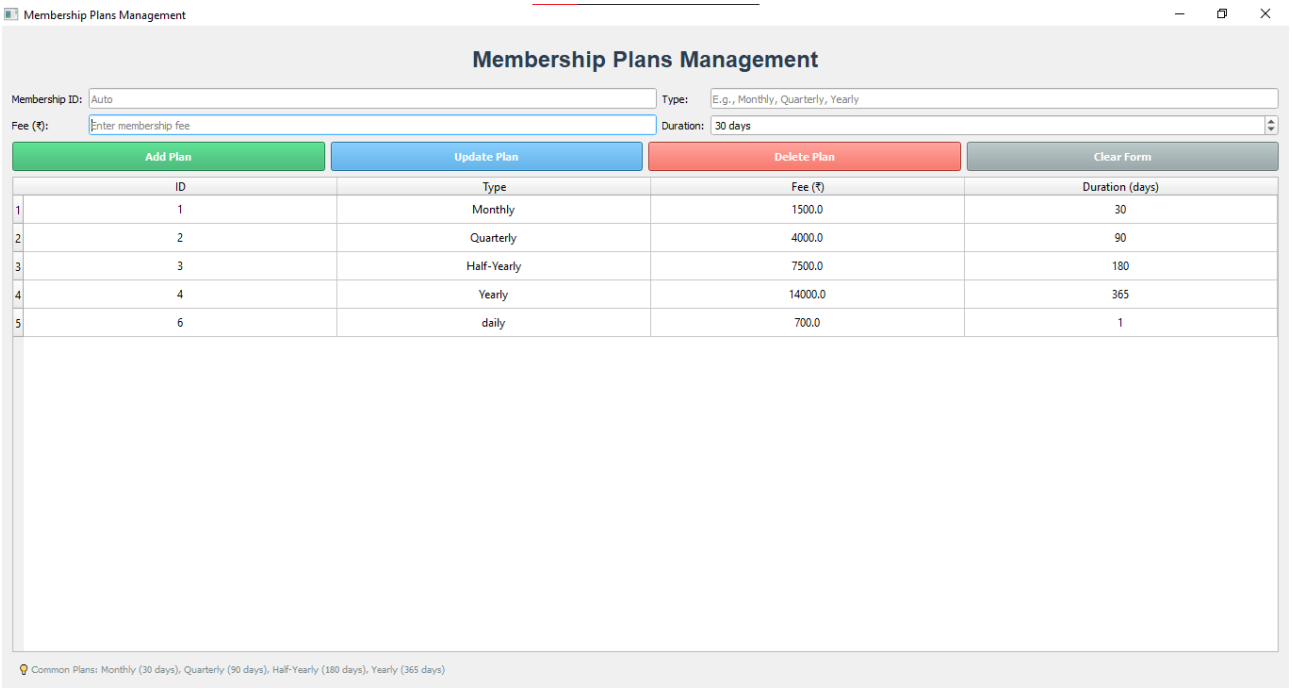


Fig. 5.9: Membership Plans Management.

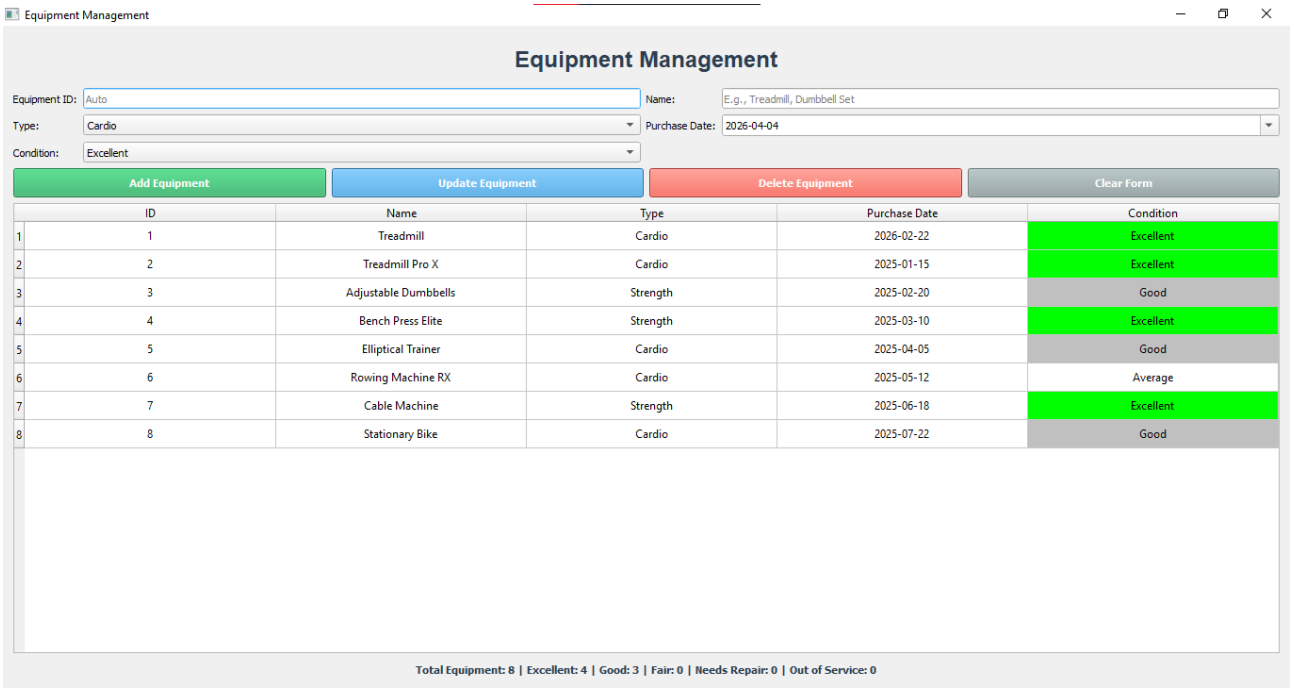


Fig. 5.10: Equipment Management.

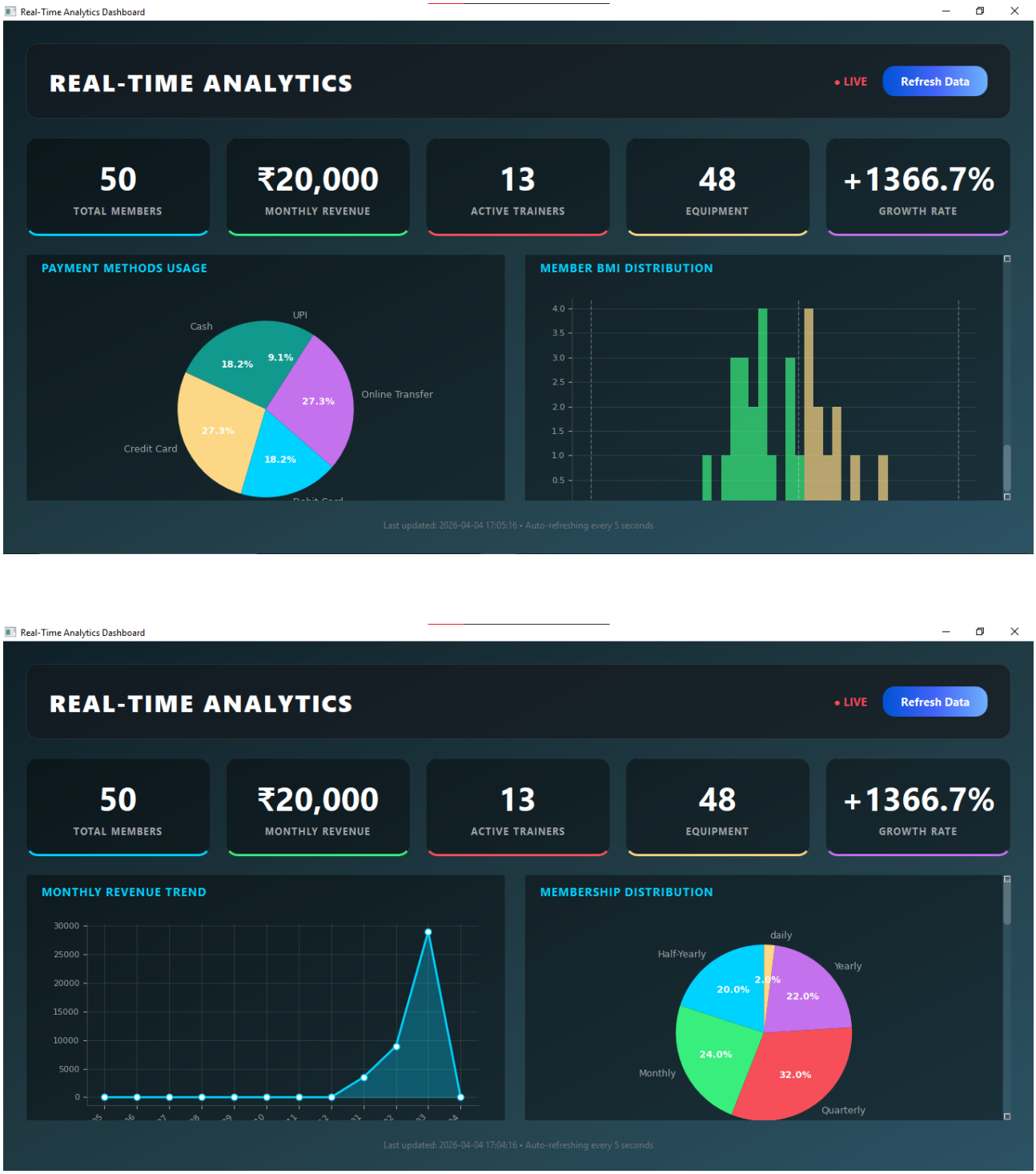


Fig. 5.11: Real-Time Analytics Dash.

CHAPTER 6

RESULT AND DISCUSSION

6.1 System Functionality Evaluation:

The functionality of the Gym Management System was evaluated by testing all modules individually as well as in an integrated environment. Each module was verified to ensure that it performs its intended tasks correctly and interacts properly with other modules.

The member management module successfully performs operations such as adding, updating, deleting, and viewing member records. It allows users to store detailed information about members in an organized manner. Validation mechanisms ensure that incorrect or incomplete data is not entered into the system, thereby maintaining data accuracy and consistency.

The trainer management module efficiently handles trainer-related information, including their specialization, contact details, and assignments. It supports proper allocation of trainers to members and ensures that data is easily accessible when required.

The membership management module enables the creation and management of different membership plans. It accurately tracks the duration and validity of memberships, helping gym staff monitor active and expired memberships. This feature plays an important role in maintaining smooth operations.

The payment module records all financial transactions and maintains a complete payment history for each member. It supports multiple payment modes and helps identify pending dues. The module ensures that financial data is accurate and up to date, which is essential for effective financial management.

The workout plan module allows trainers to create and assign customized workout plans based on individual member goals. It supports tracking of assigned plans and helps in monitoring the progress of members over time.

The metrics module records body measurements such as weight and height and automatically calculates BMI. This helps in tracking fitness progress and provides useful insights for trainers and members.

The equipment module manages gym inventory and tracks the condition of equipment. It ensures proper maintenance and efficient utilization of resources.
The analytics module provides graphical representations of data through charts and graphs. It displays information such as revenue trends, membership distribution, and growth patterns, which helps in better decision-making.

Overall, all modules function correctly and integrate seamlessly with each other. The system provides a smooth and efficient user experience, meeting all the functional requirements.

6.2 Performance Evaluation:

The performance of the Gym Management System was evaluated based on its responsiveness, efficiency, and ability to handle data operations. The system performs efficiently under normal operating conditions and provides consistent results.

The use of SQLite ensures fast data access and minimal resource consumption. Database operations such as insertion, retrieval, updating, and deletion are performed quickly without noticeable delays. The system maintains good performance even when handling a moderate amount of data.

The frontend interface is highly responsive and reacts quickly to user inputs. Actions such as form submission, button clicks, and data display are executed smoothly. The use of optimized queries and structured coding practices contributes to the overall performance of the system.

Memory usage is low, as the system is designed to run on standard desktop environments. The application does not require high-end hardware and can operate efficiently on basic systems.

Transaction management ensures data consistency and reliability. In case of errors, appropriate handling mechanisms prevent data loss or corruption. This improves the robustness of the system.

The analytics module, which includes charts and visualizations, performs efficiently without affecting the overall performance. Updates are handled smoothly, and the system remains stable.

Although the system performs well for small to medium-sized gyms, scalability may be limited for very large datasets. In such cases, upgrading to a more powerful database system may be required.

Overall, the system demonstrates good performance, reliability, and stability.

6.3 User Feedback and Adoption:

User feedback was collected to evaluate the usability and effectiveness of the system. The system was tested by users such as gym staff and administrators to understand their experience and identify areas for improvement.

Most users found the system easy to use due to its simple and intuitive interface. The clear layout and organized modules made it easy to navigate and perform tasks. Users were able to learn and operate the system quickly, even without prior technical knowledge.

The automation of tasks such as member registration, payment tracking, and report generation significantly reduced manual effort. Users reported improved efficiency and reduced workload. The system also minimized errors, leading to more accurate data management.

The dashboard and analytics features were appreciated for providing quick insights into gym operations. Users were able to monitor performance and make informed decisions based on data.

Some users suggested enhancements such as adding mobile access, online booking features, and automated notifications. These suggestions indicate that users are interested in further improvements and are willing to adopt the system in real-world scenarios.

Overall, user feedback was positive, and the system was well accepted. The ease of use, efficiency, and reliability contributed to its successful adoption.

6.4 Future Directions and Enhancements:

The Gym Management System can be further improved by adding new features and enhancing existing functionalities. Future enhancements can focus on improving accessibility, performance, and user experience.

One major enhancement is the development of a mobile or web-based version of the system. This would allow users to access the system from anywhere, improving convenience and flexibility.

Integration with cloud technology can improve data security and enable real-time synchronization. It would also provide backup and recovery options, reducing the risk of data loss.

The system can be enhanced by adding online payment options and automated billing features. This would simplify payment processing and improve financial management.

Advanced analytics and reporting tools can be implemented to provide deeper insights into gym operations. This would help in better decision-making and business growth.

Additional features such as attendance tracking, notification systems, and biometric integration can further improve automation and efficiency.

Security features such as role-based access control and data encryption can be added to protect sensitive information.

Overall, these enhancements will make the system more powerful, scalable, and suitable for larger applications.

The Gym Management System successfully meets its objectives by providing an efficient and reliable solution for managing gym operations. The system performs well in terms of functionality, performance, and usability. Positive user feedback further confirms its effectiveness. With future enhancements, the system can be expanded to meet advanced requirements and provide even greater value.

CHAPTER 7

CONCLUSION

The Gym Management System has been successfully designed and developed as a desktop-based application to automate and simplify the daily operations of a gym. The system effectively replaces traditional manual methods with a structured and computerized solution, improving efficiency, accuracy, and overall management. By integrating various modules such as member management, trainer management, membership handling, payment tracking, workout planning, equipment management, and analytics, the system provides a complete and unified platform for gym administration.

The implementation of the system demonstrates that it is capable of handling core gym operations in an organized and efficient manner. The use of Python and the PyQt5 framework ensures a user-friendly and interactive interface, while SQLite provides reliable and lightweight database management. The modular design of the system allows each component to function independently while maintaining seamless integration, making the system easy to maintain and extend.

The system has shown significant improvements over the existing manual system by reducing paperwork, minimizing human errors, and enabling faster data processing. It provides real-time access to information, allowing gym staff to manage records more effectively and make better decisions. Features such as automated calculations, data validation, and analytical reports further enhance the functionality and usability of the system.

User feedback indicates that the system is easy to use and significantly improves productivity. The intuitive interface and organized structure reduce the learning curve and make the system accessible even to users with limited technical knowledge. The inclusion of analytics and reporting features adds value by providing insights into gym operations and performance.

Although the system meets its primary objectives, there is scope for future enhancements such as cloud integration, mobile application support, advanced analytics, and improved security features. These enhancements can further expand the capabilities of the system and make it suitable for larger and more complex environments.

In conclusion, the Gym Management System is a practical, efficient, and reliable solution for managing gym operations. It demonstrates the effective use of modern technologies to solve real-world problems and provides a strong foundation for future development and improvements.

CHAPTER 8

REFERENCES

1. https://sqlite.org/docs.html
2. https://docs.python.org/3/library/sqlite3.html
3. https://www.w3schools.com/sql/
4. https://docs.python.org/
5. https://www.w3schools.com/python/
6. https://docs.pypi.org/
7. https://doc.qt.io/qtforpython-5/
8. https://pypi.org/project/PyQt5/
9. https://matplotlib.org/stable/index.html
10. https://pypi.org/project/matplotlib/